

RNA-seq: Patógenos associados a hospedeiros

Dalmolin Systems Biology Group

Table of contents

1	RNA-seq: Patógenos associados a hospedeiros	5
2	Visão geral da aula	6
3	Configuração e Descrição do Conjunto de Dados	8
3.1	Conjunto de Dados	8
3.1.1	Descrição do Experimento	8
3.1.2	Estrutura de Lotes (Batches)	8
3.1.3	Seleção de Amostras	8
4	Pré-processamento	11
5	Pré-processamento e Descontaminação	12
5.1	1. O Ciclo do Sequenciamento: Das Células às “Reads”	12
5.2	2. O Pipeline nf-core/detaxizer	12
5.3	3. Automação com Nextflow	13
5.3.1	O Script de Execução	13
5.3.2	Entendendo os Parâmetros:	14
5.3.3	Recursos Esperados	14
5.4	Visualizando a Composição Taxonômica (Kraken2)	15
6	Quantificação	17
6.1	1. Alinhamento e Quantificação	17
6.2	2. O Pipeline nf-core/rnaseq	18
6.2.1	O Script de Execução	18
6.2.2	Entendendo os Parâmetros:	19
6.2.3	Recursos Esperados	19
6.3	3. Fontes de Dados: Ensembl Fungi	19
6.3.1	4. Entendendo os Outputs: Do Transcrito ao Gene à Proteína	19
7	Análise de Expressão Diferencial	21
7.1	Sanidade de Dados	21
7.1.1	1. Carregando os Pacotes	21
7.1.2	2. Importando a Matriz de Contagens	22
7.1.3	3. Importando os Metadados	22
7.1.4	4. Preparando a Matriz de Contagens	25

7.1.5	5. Criando o Objeto DESeq2	25
7.1.6	6. Análise de Componentes Principais (PCA)	26
7.1.7	7. Scree Plot	27
7.1.8	8. Clusterização Hierárquica	29
7.1.9	9. Análise de Covariáveis	31
7.2	Análise de Expressão Diferencial de Genes (DGE)	33
7.2.1	10. Rodando o DESeq2	33
7.2.2	11. Extraíndo os Resultados	34
7.2.3	12. Definindo os Genes Diferencialmente Expressos	35
7.2.4	13. MA Plot	36
7.2.5	14. Volcano Plot	38
7.2.6	15. Heatmap dos Genes Diferencialmente Expressos	40
8	Análises Downstream	43
9	Enriquecimento Funcional	44
9.1	O que são GO e KEGG?	44
9.1.1	Gene Ontology (GO)	44
9.1.2	KEGG Pathways	45
9.1.3	Principais diferenças	45
9.2	A Importância dos Bancos de Dados Curados	45
9.3	1. Carregando os Pacotes	46
9.4	2. Carregando os Resultados da Expressão Diferencial	46
9.5	3. Preparando os Conjuntos de Genes	47
9.6	4. Conversão de Identificadores para o KEGG	48
9.7	5. Enriquecimento KEGG — GSEA	49
9.7.1	Construindo o vetor ranqueado	49
9.7.2	Rodando gseKEGG()	50
9.7.3	Dotplot — GSEA KEGG (UP vs. DOWN)	50
9.7.4	Ridgeplot — distribuição de log2FC por via	53
9.7.5	Curva de Running Enrichment Score — Top 3 vias	54
9.7.6	Barplot Bidirecional — NES por via	55
9.7.7	Como interpretar os gráficos:	56
9.8	6. Enriquecimento GO com g:Profiler	57
9.9	7. Sobre o pacote g:Profiler	57
9.9.1	Carregando o pacote gprofiler2	57
9.9.2	Dotplot — GO BP Upregulados	59
9.9.3	Dotplot — GO BP Downregulados	60
9.9.4	Dotplot — GO BP Todos os DEGs	61
9.9.5	Barplot Bidirecional — GO BP UP vs. DOWN	62
9.9.6	Barplot — GO BP Todos os DEGs	64
9.9.7	Como interpretar os gráficos:	65
9.10	8. Resumo e Interpretação	66

10 Redes de Interação Proteína-Proteína (PPI)	68
10.1 9. O Banco de Dados STRING	68
10.2 10. Carregando os Pacotes	69
10.3 11. Preparando a Lista de Genes	70
10.4 12. Consultando o STRING via API	71
10.4.1 Mapeando os identificadores	71
10.4.2 Obtendo a rede de interações	71
10.5 13. Filtrando as Interações com combinescores	72
10.6 14. Construindo o Grafo com igraph	74
10.6.1 Anotando os nós com informações de expressão	75
10.7 15. Layout Interativo com easylayout	75
10.8 16. Visualização da Rede PPI com ggraph	76
10.8.1 Como interpretar a rede PPI:	78

1 RNA-seq: Patógenos associados a hospedeiros

2 Visão geral da aula

Licença: [Creative Commons Attribution Share Alike 4.0 International License](#)

Público-alvo: Pesquisadores, estudantes de pós-graduação e bioinformatas interessados em transcriptômica de interação patógeno-hospedeiro.

Nível: Intermediário

Pré-requisitos Para acompanhar este curso, os alunos devem ter conhecimentos em:

1. **R Básico:** Capacidade de carregar bibliotecas, manipular data frames e executar scripts.
2. **Biologia Molecular:** Conceitos fundamentais de RNA-seq, expressão gênica e biologia de fungos/mamíferos.
3. **Ambiente:** Familiaridade com o uso do RStudio.

Descrição Este repositório contém os materiais para o curso “**RNA-seq: Patógenos associados a hospedeiros**”, com foco no modelo de *Candida albicans* infectando macrófagos de *Mus musculus* tratados com vancomicina. Organizado pelo Dalmolin’s Systems Biology Group (BioME-UFRN), o workshop aborda as etapas críticas de limpeza de dados contaminados pelo hospedeiro utilizando o **nf-core/detaxizer**, a quantificação de leituras do patógeno com o **nf-core/rnaseq**, e a execução de análise de expressão diferencial e enriquecimento funcional.

A vancomicina é um antibiótico amplamente prescrito para o tratamento de infecções bacterianas Gram-positivas. Já foi mostrado que este antibiótico pode romper as respostas imunológicas antifúngicas protetoras via disbiose do microbioma, aumentando a suscetibilidade à candidíase invasiva. Antibióticos são um fator de risco independente para o desenvolvimento desta infecção fúngica potencialmente fatal, mas não está claro se mecanismos independentes da microbiota também impulsionam esta associação. Aqui, comparamos as respostas transcricionais de macrófagos derivados da medula óssea de camundongos em um ponto temporal inicial pós-infecção (2 horas), comparando com a linha de base, usando macrófagos não tratados ou tratados com vancomicina. Os dados brutos deste estudo estão disponíveis sob o BioProject **PRJNA1120019**.

Importante: Para as análises realizadas neste curso, estamos considerando apenas as amostras que foram estimuladas com *Candida albicans* (conforme @data/SraRunTable.csv).

Resultados de Aprendizagem: Ao final do curso, os alunos serão capazes de:

1. **Compreender** a descontaminação *in silico* de leituras do hospedeiro através do pipeline **nf-core/detaxizer**. [Compreender]

2. **Delinear** a configuração e execução do pipeline nf-core/rnaseq especificamente para a quantificação de patógenos fúngicos. [Aplicar]
3. **Realizar** análise de expressão diferencial (DGE) utilizando **DESeq2** para identificar genes-chave no processo de infecção. [Analisar]
4. **Interpretar** assinaturas biológicas por meio de análise de enriquecimento funcional (GO/KEGG). [Avaliar]
5. **Visualizar** dados complexos através de PCA, heatmaps e volcano plots para interpretação biológica. [Criar]

Requisitos: Os participantes devem ter acesso a um notebook com R e RStudio instalados para a etapa de análise estatística.

Agradecimentos:

- Dalmolin Systems Biology Group
- Bioinformática Multidisciplinar (BioME - IMD/UFRN)
- Programa de Pós-Graduação em Bioinformática (PPg-Bioinfo - UFRN)

3 Configuração e Descrição do Conjunto de Dados

3.1 Conjunto de Dados

Os dados utilizados nestas análises provêm do BioProject **PRJNA1120019**. Este estudo investiga os mecanismos pelos quais o antibiótico vancomicina afeta a resposta imune antifúngica em macrófagos (para mais detalhes, consulte o [artigo original publicado na mBio](#)).

3.1.1 Descrição do Experimento

- **Organismo:** *Mus musculus* (Macrófagos derivados da medula óssea - BMDMs)
- **Patógeno:** *Candida albicans* (cepa SC5413)
- **Tratamento:** Vancomicina vs. Não tratado (Controle)
- **Ponto Temporal:** 2 horas pós-infecção

O desenho experimental envolveu a diferenciação de macrófagos na presença ou ausência de vancomicina, seguida de infecção por *Candida albicans*.

3.1.2 Estrutura de Lotes (Batches)

O experimento foi conduzido em **6 lotes independentes** (`experiment_group` 1 a 6). Cada lote contém: 4 tratadas com vancomicina e 4 controles (não tratada). Essa replicação em lotes é importante porque variações técnicas entre dias de cultivo, preparo de reagentes ou extração de RNA podem introduzir diferenças que não são biológicas. Na etapa de Análise de Expressão Diferencial, veremos como levar esses lotes em consideração no modelo estatístico para não confundir efeito de *batch* com efeito do tratamento.

3.1.3 Seleção de Amostras

Para as análises realizadas ao longo deste curso, selecionamos especificamente as amostras estimuladas com *Candida albicans*. É importante notar que cada amostra biológica pode conter múltiplos arquivos de sequenciamento (Run SRR), os quais devem ser combinados para a análise final.

A tabela abaixo apresenta as amostras consideradas, extraídas do arquivo @data/SraRunTable.csv:

Run	stimulus	treatment	cell_type
SRR29279085	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279093	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279101	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279109	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279117	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279125	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279133	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279141	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279149	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279157	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279165	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279173	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279086	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279087	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279088	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279094	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279095	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279096	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279102	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279103	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279104	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279110	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279111	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279112	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279118	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279119	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279120	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279126	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279127	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279128	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279134	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279135	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279136	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279142	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279143	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279144	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279150	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279151	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279152	Candida albicans	Vancomycin	Bone-marrow Macrophage

Run	stimulus	treatment	cell_type
SRR29279158	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279159	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279160	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279166	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279167	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279168	Candida albicans	Vancomycin	Bone-marrow Macrophage
SRR29279174	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279175	Candida albicans	Untreated (control)	Bone-marrow Macrophage
SRR29279176	Candida albicans	Untreated (control)	Bone-marrow Macrophage

Nosso interesse aqui será ver a expressão de *Candida albicans* frente a exposição de vancomicina, em macrófagos.

4 Pré-processamento

Esta seção do curso foca na preparação dos dados brutos. Como discutimos, em experimentos de interação patógeno-hospedeiro (como *Candida* em macrófagos), a maior parte das sequências obtidas pertencerá ao hospedeiro. O pré-processamento é a etapa onde garantimos que estamos analisando apenas o que nos interessa: o transcriptoma do patógeno.

5 Pré-processamento e Descontaminação

Nesta etapa teórica, vamos entender como saímos das amostras biológicas para os arquivos de sequências limpas, prontos para a quantificação.

5.1 1. O Ciclo do Sequenciamento: Das Células às “Reads”

O processo começa com a extração do RNA total da amostra (macrófagos infectados). Esse RNA é convertido em cDNA e fragmentado para a construção de uma **biblioteca de sequenciamento**.

```
@Sample Label
AATGATACGGCGTATGTATTCTCTCTATTTTGC GAAGTCCAATGGTGCGTATATGCC TTCTGCTTGTC
+
{:FFFFFFF,FFHHHF4FFFFFF<FED@FFFF:F,,FFFF9C;=?FFF?2AADDFF:FF:FF?GBB?<F99FPFF
```

Figure 5.1: Um exemplo de uma sequência de um arquivo FASTQ, com os índices de qualidade abaixo

As máquinas de sequenciamento (como as da Illumina) leem esses fragmentos e geram arquivos no formato **FASTQ**. Cada arquivo FASTQ contém milhões de sequências curtas chamadas **reads**, acompanhadas de índices de qualidade (Phred Score) para cada base identificada. No entanto, esses arquivos brutos contêm:

- Adaptadores de sequenciamento.
- Bases de baixa qualidade nas extremidades.
- **Contaminação do Hospedeiro:** Em modelos de infecção, até 95-99% das reads podem ser do hospedeiro (*Mus musculus*), mascarando o sinal da *Candida albicans*.

5.2 2. O Pipeline nf-core/detaxizer

Para resolver o problema da contaminação, utilizamos o pipeline [nf-core/detaxizer](#). Ele é projetado especificamente para verificar a presença de táxons específicos e filtrá-los de arquivos FASTQ.

O fluxo de trabalho do detaxizer segue estas etapas principais:

1. **Controle de Qualidade (FastQC):** Avaliação inicial da qualidade das reads brutas.
2. **Pré-processamento Opcional (fastp):** Corte de adaptadores e filtragem de reads por tamanho ou qualidade.
3. **Classificação Taxonômica (Kraken2 e/ou bbdduk):** As reads são comparadas contra bancos de dados genômicos para identificar sua origem. O **Kraken2** usa k-mers para uma atribuição rápida, enquanto o **bbduk** pode ser usado para pareamento direto contra sequências de referência.
4. **Validação Opcional (BLASTN):** Para maior rigor, as reads classificadas podem ser validadas via alinhamento BLAST contra bancos de dados de nucleotídeos.
5. **Filtragem:** O pipeline remove as reads associadas ao táxon indesejado (neste caso, *Mammalia*).
6. **Relatório (MultiQC):** Consolida todos os resultados em um único arquivo HTML, permitindo visualizar quantas reads foram removidas e a qualidade final dos dados.

5.3 3. Automação com Nextflow

A execução de todas essas ferramentas manualmente seria propensa a erros e difícil de reproduzir. Para isso, utilizamos o **Nextflow**, um motor de fluxo de trabalho que gerencia a instalação de ferramentas (via Docker/Singularity) e a execução paralela das tarefas.

i Ponto Importante

Para utilizar o Nextflow, é necessário ter o Java instalado e o comando `curl -s https://get.nextflow.io | bash`. O Nextflow organiza cada ferramenta em “processos” que são conectados em uma “pipeline”.

5.3.1 O Script de Execução

Para o nosso conjunto de dados de *Candida* e macrófagos, o comando utilizado para a descontaminação foi:

```
nextflow run nf-core/detaxizer \
  -profile docker \
  --input samplesheet.csv \
  --preprocessing \
  --kraken2db db/k2_pluspf_20260226.tar.gz \
  --classification_kraken2_post_filtering \
  --filter_trimmed \
  --save_intermediates \
  --classification_bbdduk \
```

```
--classification_kraken2 \  
--tax2filter 'Mammalia' \  
--outdir results/decontaminated/
```

5.3.2 Entendendo os Parâmetros:

- `-profile docker`: Garante que todas as ferramentas (Kraken2, FastQC, etc.) rodem dentro de containers, sem necessidade de instalação manual.
- `--tax2filter 'Mammalia'`: O ponto crucial. Instruímos o pipeline a identificar e remover tudo o que for classificado como mamífero (o hospedeiro *Mus musculus* e qualquer outra leitura humana).
- `--kraken2db`: Aponta para o banco de dados taxonômico usado para a identificação.
- `--filter_trimmed`: Aplica a filtragem nas reads já processadas pelo `fastp`.

5.3.3 Recursos Esperados

- **Tempo Total**: 17 h 45 m 53 s
- **Tempo de CPU**: 35.2 h
- **Memória Máxima**: 104.7 GB

Tip

Sobre o Banco de Dados Kraken2 (PlusPF)

O banco de dados utilizado, `k2_pluspf`, é a versão Standard plus Refseq protozoa & fungi. Ele contém o genoma humano, genomas de bactérias, archaea, vírus e, crucialmente para este curso, o RefSeq de protozoários e fungos.

Isso é fundamental porque, para filtrar o que é *Mammalia* com segurança, o software precisa saber diferenciar o que é hospedeiro do que é patógeno. Sem as sequências de fungos no banco, o Kraken2 poderia classificar erroneamente reads da *Candida* como sendo de origem desconhecida ou até atribuí-las falsamente a outros grupos. Para mais informações sobre as referências do Kraken2, consulte <https://benlangmead.github.io/aws-indexes/k2>

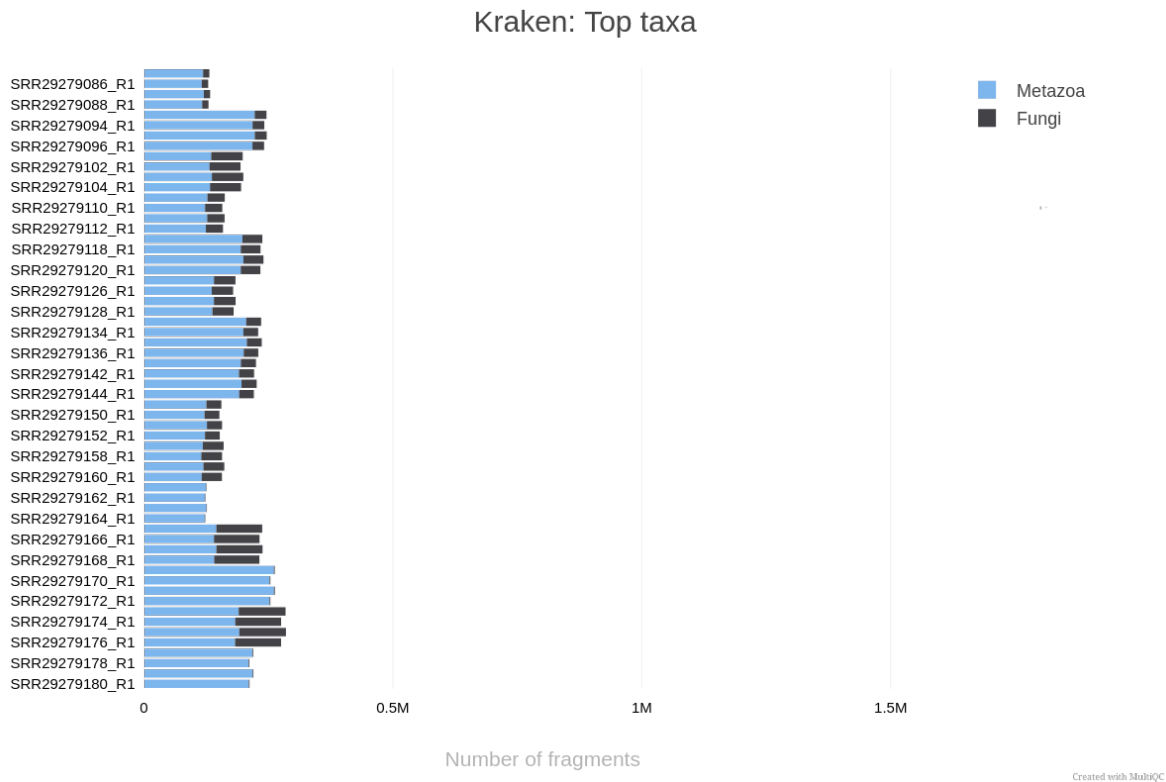
Ao final deste processo, obtemos arquivos FASTQ “limpos”, contendo predominantemente sequências da *Candida albicans*, prontos para a próxima etapa: a **Quantificação**.

Ponto Importante

Uma estratégia igualmente válida seria remover as leituras de *Fungi* e analisar a partir das leituras removidas ao invés das filtradas. Talvez um de vocês possa tentar isso depois!

5.4 Visualizando a Composição Taxonômica (Kraken2)

Para avaliar a eficiência da nossa estratégia de descontaminação e entender a composição das nossas amostras brutas, analisamos a distribuição taxonômica das reads antes da filtragem. O gráfico abaixo, gerado pelo MultiQC, resume os principais táxons identificados pelo Kraken2.



Como interpretar este gráfico:

- Predomínio de Metazoa (Azul): Como esperado em um modelo de infecção de macrófagos, a grande maioria dos fragmentos é classificada como Metazoa (neste caso, representando o hospedeiro, *Mus musculus*). Isso ilustra o desafio da “contaminação do hospedeiro”: o sinal do camundongo é ordens de grandeza maior que o do patógeno.
- O Nosso Alvo - Fungi (Cinza Escuro): Os segmentos em cinza escuro representam o reino Fungi, especificamente a *Candida albicans*. Observe que a proporção de reads fúngicas varia entre as amostras.

 Tip

Por que isso é importante?

Ao utilizarmos o pipeline `nf-core/detaxizer` com o parâmetro `-tax2filter 'Mammalia'`, estamos essencialmente “removendo” a parte azul dessas barras e mantendo apenas a parte cinza escuro. Isso garante que, na etapa de Quantificação, o software não perca tempo processando sequências do hospedeiro que não nos interessam para este estudo específico do patógeno.

Com os dados descontaminados, como podemos saber exatamente quantos genes da *Candida* estão presentes em cada amostra? Veremos isso na próxima seção sobre o `nf-core/rnaseq`.

6 Quantificação

Com os dados descontaminados, o próximo passo é identificar a quais genes da *Candida albicans* essas sequências pertencem e contar quantas vezes cada gene aparece em nossas amostras. Este processo é conhecido como **Quantificação**.

6.1 1. Alinhamento e Quantificação

O processo de quantificação geralmente envolve duas etapas principais:

1. **Alinhamento (Mapping):** As reads (sequências curtas) são comparadas contra um genoma de referência ou transcriptoma. É como montar um quebra-cabeça onde usamos o genoma como o guia para saber de onde cada peça veio.
2. **Contagem (Counting):** Uma vez que sabemos onde a read se encaixa, verificamos se ela caiu em uma região anotada como um “gene”. O resultado final é uma **Matriz de Contagens**, onde cada linha é um gene e cada coluna é uma amostra.

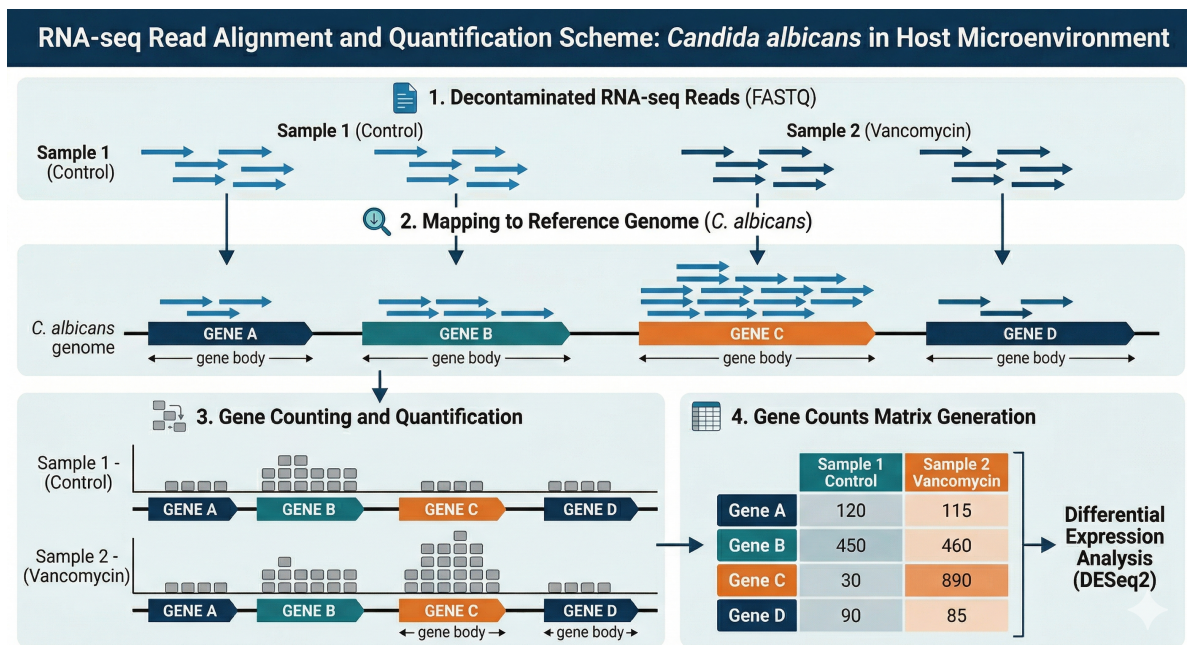


Figure 6.1: Esquema simplificado de alinhamento de reads contra um genoma de referência e a geração da matriz de contagens.

6.2 2. O Pipeline nf-core/rnaseq

Para esta etapa, utilizamos o [nf-core/rnaseq](#), um dos pipelines mais robustos e utilizados no mundo para análise de transcriptômica. Ele automatiza desde o controle de qualidade final até a geração das matrizes de expressão.

6.2.1 O Script de Execução

O comando utilizado para quantificar o transcriptoma da *Candida albicans* foi:

```
nextflow run nf-core/rnaseq \
  --input rnaseq_samplesheet.csv \
  --outdir rnaseq_results \
  --gtf db/Candida_albicans.GCA000182965v3.62.gtf.gz \
  --fasta db/Candida_albicans.GCA000182965v3.dna.toplevel.fa.gz \
  --profile docker \
  --resume --with-tower
```

6.2.2 Entendendo os Parâmetros:

- **--fasta:** O arquivo de sequência genômica (o “dicionário” de DNA) da *Candida*.
- **--gtf:** O arquivo de anotação, que diz ao software as coordenadas exatas de onde cada gene começa e termina no genoma.
- **-with-tower:** Permite o monitoramento remoto da execução via [Nextflow Tower](#).

6.2.3 Recursos Esperados

- **Tempo Total:** 3 h 12 m 13 s
- **Tempo de CPU:** 113.0 h
- **Memória Máxima:** 16 GB

6.3 3. Fontes de Dados: Ensembl Fungi

Para organismos não-modelos ou patógenos fúngicos, a fonte para genomas e anotações é o [Ensembl Fungi](#).

Para este curso, utilizamos a linhagem de referência **SC5314** da *Candida albicans*. As referências foram adquiridas nos seguintes links:

- **Genoma (Fasta):** [C. albicans DNA Toplevel](#)
- **Anotação (GTF):** [C. albicans Gene Annotation](#)

! Por que usar o Ensembl Fungi?

Diferente do Ensembl principal (focado em vertebrados), o Ensembl Fungi mantém curadoria específica para as complexidades de genomas fúngicos, como estruturas de íntrons compactas e polimorfismos frequentes, garantindo que a nossa quantificação seja a mais precisa possível.

6.3.1 4. Entendendo os Outputs: Do Transcrito ao Gene à Proteína

Antes de abrirmos o R para a Análise de Expressão Diferencial, precisamos alinhar exatamente o que o RNA-Seq mede e como isso se conecta com o resto do curso.

Lembrando o dogma central da biologia molecular (**DNA → RNA → Proteína**), nosso dado é um “snapshot” do **transcriptoma** (RNA). O Salmon quantifica **transcritos** (mRNA), mas nas próximas etapas falaremos constantemente em **genes** e **proteínas**. Por que essa mudança de termos?

- **Dos Transcritos aos Genes:** Um único gene pode gerar múltiplos transcritos (isoformas) por *splicing* alternativo. Para simplificar a análise, especialmente no genoma mais compacto da *Candida albicans*, o padrão é **somar as contagens das isoformas em um único valor por gene**. O pipeline *nf-core/rnaseq* já fez isso para nós e gerou o arquivo que usaremos: `salmon.merged.gene_counts.tsv`.
- **Dos Genes às Proteínas:** No final da nossa análise, usaremos esses genes para mapear vias biológicas e redes de interação (PPI).

i Atenção aos Termos

Na literatura, é muito comum ler “o gene X está superexpresso” ou “a proteína Y alterou” em estudos de RNA-Seq. Na prática, nós medimos apenas a **abundância de transcritos** e a usamos como um *proxy* (uma estimativa) da atividade do gene e da futura tradução proteica. Seguiremos essa mesma convenção prática ao longo do curso.

Com a nossa matriz de contagens pronta e os conceitos alinhados, vamos abrir o R. A grande pergunta agora é: **quais genes alteraram sua expressão devido ao tratamento com vancomicina?** É o que vamos descobrir a seguir usando o DESeq2.

7 Análise de Expressão Diferencial

Nas etapas anteriores, descontaminamos as reads do hospedeiro (*Mus musculus*) e quantificamos o transcriptoma de *Candida albicans* com o nf-core/rnaseq. O resultado final desse pipeline é uma **matriz de contagens**, onde cada linha representa um gene e cada coluna uma amostra. A partir dessa matriz, podemos realizar a análise de expressão diferencial para identificar quais genes da *Candida* foram regulados (upregulados ou downregulados) em resposta ao tratamento com vancomicina.

7.1 Sanidade de Dados

Antes de realizarmos a análise de expressão diferencial propriamente dita, é fundamental explorar a estrutura do nosso conjunto de dados. Essa etapa, frequentemente chamada de sanidade de dados ou análise exploratória, serve para responder perguntas essenciais:

- As amostras se agrupam conforme esperamos (por tratamento)?
- Existem *outliers* ou amostras com comportamento atípico?
- Quais variáveis do metadado influenciam a expressão gênica?

Para tal, precisamos de técnicas que consigam lidar com a **multidimensionalidade** dos dados: estamos analisando milhares de genes simultaneamente. Técnicas como a Análise de Componentes Principais (PCA) e a clusterização hierárquica nos permitem “resumir” essa complexidade e visualizar padrões.

7.1.1 1. Carregando os Pacotes

Vamos começar carregando os pacotes que utilizaremos ao longo de toda esta seção prática.

```
suppressPackageStartupMessages({  
  library(DESeq2)  
  library(PCAtools)  
  library(dplyr)  
  library(readr)
```

```
library(tibble)
library(ggplot2)
library(pheatmap)
})
```

7.1.2 2. Importando a Matriz de Contagens

O nf-core/rnaseq com o quantificador Salmon gera uma tabela consolidada com as contagens de todos os genes por amostra. Vamos carregá-la:

```
# Importar matriz de contagens (gerada pelo Salmon via nf-core/rnaseq)
gene_counts <-
  read_tsv(here::here("data/rnaseq/salmon.merged.gene_counts.tsv"),
           show_col_types = FALSE)

# Visualizar as primeiras linhas e colunas
gene_counts[1:5, 1:6]
```

```
# A tibble: 5 x 6
  gene_id      gene_name SRR29279085 SRR29279086 SRR29279087 SRR29279088
  <chr>      <chr>      <dbl>      <dbl>      <dbl>      <dbl>
1 C1_00010W_A C1_00010W_A      0          0          0          0
2 C1_00020C_A C1_00020C_A      2          4          1          2
3 C1_00030C_A C1_00030C_A      0          0          0          0
4 C1_00040W_A CTA2             0          0          0          0
5 C1_00050C_A C1_00050C_A      0          0          0          0
```

A primeira coluna (`gene_id`) contém os identificadores dos genes de *Candida albicans* no Ensembl Fungi, e a segunda (`gene_name`) contém o nome do gene quando disponível. As demais colunas são as amostras, identificadas pelo código SRR.

7.1.3 3. Importando os Metadados

As informações sobre as condições experimentais de cada amostra estão no arquivo `SraRunTable.csv`, obtido do SRA. Precisamos filtrá-lo para manter apenas as amostras que foram estimuladas com *Candida albicans*. Vamos organizar o metadado para que ele possa ser facilmente integrado com a matriz de contagens. O DESeq2 exige que as colunas da matriz de contagens correspondam às linhas do metadado, e que o metadado contenha as variáveis que queremos testar (neste caso, o tratamento) e as covariáveis (como o grupo experimental).

Além da variável de interesse (`treatment`), nosso metadado contém uma informação crucial: o **lote experimental** (`experiment_group`). Como vimos na descrição do conjunto de dados, o experimento foi conduzido em **6 lotes independentes**, cada um contendo 4 amostras tratadas com vancomicina e 4 controles. Diferenças entre lotes, como variações no dia de cultivo, preparo de reagentes ou condições de extração, podem introduzir variabilidade que **não é interessante para o objetivo do estudo**. Se não levarmos isso em conta, corremos o risco de confundir efeito de *batch* com efeito do tratamento, o que pode gerar tanto falsos positivos quanto falsos negativos nos resultados da expressão diferencial.

```
# Importar metadados e filtrar amostras estimuladas com C. albicans
sra_table <- read_csv(here::here("data/SraRunTable.csv"), show_col_types =
  ↪ FALSE)

metadata <- sra_table %>%
  filter(stimulus == "Candida albicans") %>%
  dplyr::select(Run, treatment, experiment_group) %>%
  mutate(treatment = factor(treatment,
                            levels = c("Untreated (control)", "Vancomycin")),
         experiment_group = factor(experiment_group,
                                    levels = c(1, 2, 3, 4, 5, 6))
         ) %>%

  column_to_rownames("Run")

metadata[order(metadata$experiment_group),]
```

	treatment	experiment_group
SRR29279165	Vancomycin	1
SRR29279173	Untreated (control)	1
SRR29279166	Vancomycin	1
SRR29279167	Vancomycin	1
SRR29279168	Vancomycin	1
SRR29279174	Untreated (control)	1
SRR29279175	Untreated (control)	1
SRR29279176	Untreated (control)	1
SRR29279149	Vancomycin	2
SRR29279157	Untreated (control)	2
SRR29279150	Vancomycin	2
SRR29279151	Vancomycin	2
SRR29279152	Vancomycin	2
SRR29279158	Untreated (control)	2
SRR29279159	Untreated (control)	2
SRR29279160	Untreated (control)	2

SRR29279133	Vancomycin	3
SRR29279141	Untreated (control)	3
SRR29279134	Vancomycin	3
SRR29279135	Vancomycin	3
SRR29279136	Vancomycin	3
SRR29279142	Untreated (control)	3
SRR29279143	Untreated (control)	3
SRR29279144	Untreated (control)	3
SRR29279117	Vancomycin	4
SRR29279125	Untreated (control)	4
SRR29279118	Vancomycin	4
SRR29279119	Vancomycin	4
SRR29279120	Vancomycin	4
SRR29279126	Untreated (control)	4
SRR29279127	Untreated (control)	4
SRR29279128	Untreated (control)	4
SRR29279101	Vancomycin	5
SRR29279109	Untreated (control)	5
SRR29279102	Vancomycin	5
SRR29279103	Vancomycin	5
SRR29279104	Vancomycin	5
SRR29279110	Untreated (control)	5
SRR29279111	Untreated (control)	5
SRR29279112	Untreated (control)	5
SRR29279085	Vancomycin	6
SRR29279093	Untreated (control)	6
SRR29279086	Vancomycin	6
SRR29279087	Vancomycin	6
SRR29279088	Vancomycin	6
SRR29279094	Untreated (control)	6
SRR29279095	Untreated (control)	6
SRR29279096	Untreated (control)	6

i Ponto Importante

Repare que cada amostra biológica (**experiment_group**) possui 4 runs de sequenciamento (SRR). No nosso caso, o nf-core/rnaseq quantificou cada run individualmente. Isso significa que temos **réplicas técnicas** que devem ser levadas em consideração. Poderíamos agregá-las somando as contagens, mas como o DESeq2 lida naturalmente com o modelo de expressão, vamos mantê-las separadas e incluir o **experiment_group** no design para absorver essa variabilidade.

7.1.4 4. Preparando a Matriz de Contagens

Agora precisamos transformar a tabela importada em uma matriz numérica compatível com o DESeq2, e garantir que apenas as amostras de interesse (com estímulo por *Candida*) estejam presentes.

```
# Transformar em matriz numérica com gene_id como rownames
count_matrix <- gene_counts %>%
  dplyr::select(gene_id, all_of(rownames(metadata))) %>%
  column_to_rownames("gene_id") %>%
  as.matrix()

# O DESeq2 requer valores inteiros
count_matrix <- round(count_matrix)

# Verificar correspondência entre metadado e contagens
identical(colnames(count_matrix), rownames(metadata))
```

```
[1] TRUE
```

```
dim(count_matrix)
```

```
[1] 6468 48
```

Temos **6468 genes** e **48 amostras** prontos para análise.

7.1.5 5. Criando o Objeto DESeq2

O DESeq2 trabalha com um objeto especial (`DESeqDataSet`) que armazena juntos a matriz de contagens, o metadado e o modelo estatístico. Aqui, nosso modelo inclui o `experiment_group` como covariável e o `treatment` como a variável de interesse:

```
dds <- DESeqDataSetFromMatrix(countData = count_matrix,
                              colData = metadata,
                              design = ~ experiment_group + treatment)
```

Antes de prosseguirmos, vamos aplicar a **Variance Stabilizing Transformation (VST)**. Dados de RNA-seq possuem uma característica chamada **heteroscedasticidade**: a variância das contagens aumenta conforme a média de expressão cresce (ou seja, genes altamente expressos apresentam maior variância absoluta). Por outro lado, se aplicássemos apenas uma transformação logarítmica simples, os genes de baixa expressão ficariam com uma variância

artificialmente inflada. A transformação VST corrige esse comportamento, tornando a variância mais homogênea em todos os níveis de expressão. Isso é essencial para análises exploratórias, como a PCA e a clusterização, mas *não* para o teste de expressão diferencial em si (que exige as contagens brutas não normalizadas).

```
dds_vst <- varianceStabilizingTransformation(dds)
counts_vst <- assay(dds_vst)
```

7.1.6 6. Análise de Componentes Principais (PCA)

A PCA é uma técnica de **redução de dimensionalidade** que projeta os milhares de genes em um número menor de componentes. Cada componente principal captura um eixo de variabilidade nos dados:

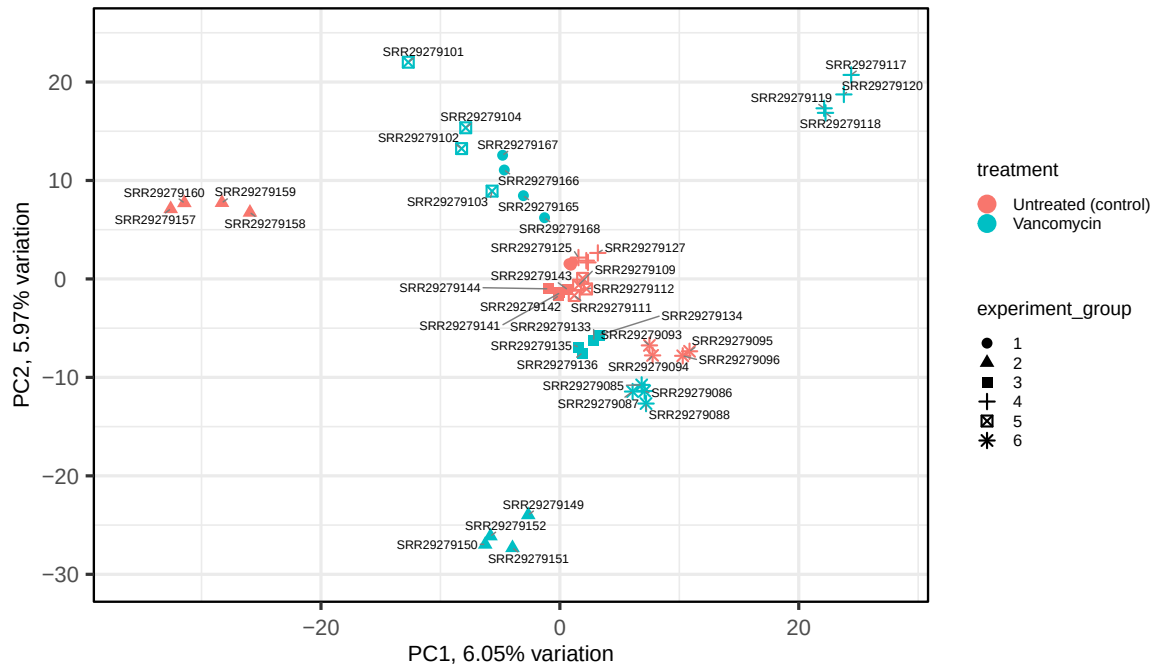
- **PC1** captura a *maior* fonte de variação.
- **PC2** captura a segunda maior, e assim por diante.

Se o tratamento com vancomicina realmente altera o perfil transcricional da *Candida*, esperamos ver as amostras se separarem por grupo ao longo de uma das primeiras componentes.

```
pca_result <- pca(counts_vst, metadata = metadata)

biplot(pca_result,
       colby = "treatment",
       shape = "experiment_group",
       legendPosition = "right",
       title = "PCA - Transcriptoma de C. albicans")
```

PCA – Transcriptoma de *C. albicans*



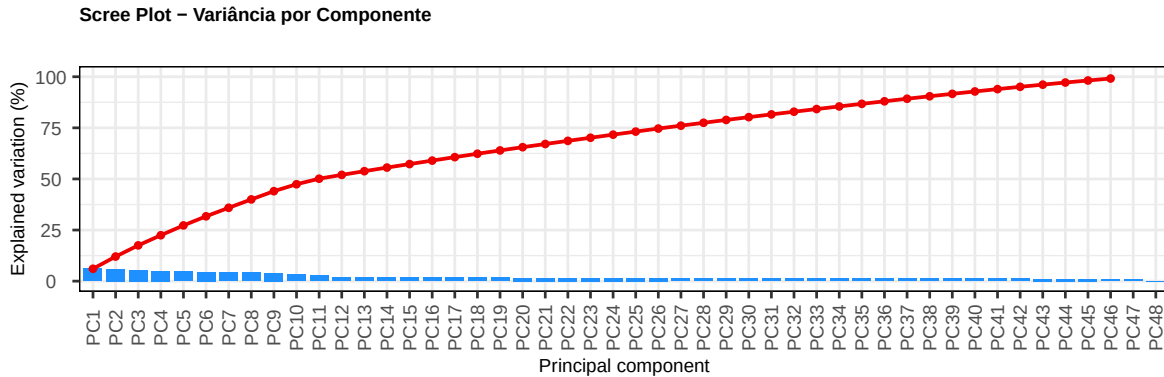
7.1.6.1 Como interpretar o biplot:

- **Proximidade entre pontos** indica similaridade no perfil de expressão global. Amostras do mesmo grupo de tratamento devem se agrupar.
- **Eixos PC1 e PC2:** o percentual ao lado de cada eixo indica quanto da variância total aquela componente explica. Se PC1 sozinha explica, por exemplo, 40% da variância, ela é a dimensão dominante do seu dado.
- A **forma** dos pontos diferencia os grupos experimentais (réplicas biológicas), o que permite avaliar se a variação entre réplicas é menor do que a variação entre tratamentos — o cenário ideal.

7.1.7 7. Scree Plot

O **scree plot** mostra a proporção de variância explicada por cada componente principal. Ele ajuda a responder: *quantas dimensões são necessárias para descrever a variabilidade do meu dado?*

```
screepLOT(pca_result,
          components = getComponents(pca_result),
          title = "Scree Plot - Variância por Componente")
```



💡 Como interpretar o scree plot

Procure pelo ponto onde a curva se achata (*elbow*). Geralmente, em dados de RNA-seq, as 2–4 primeiras componentes são suficientes para descrever a maior parte da variabilidade. No nosso caso, é normal que as duas primeiras componentes não expliquem uma fração muito alta da variância. Isso acontece porque temos **6 lotes experimentais independentes**, cada um introduzindo sua própria fonte de variabilidade. A variância total acaba sendo “fragmentada” entre muitas componentes, cada uma capturando a variação de um ou mais lotes. Se analisássemos um único lote, veríamos PC1 e PC2 explicando uma proporção bem maior.

7.1.7.1 PCA dentro de um único lote

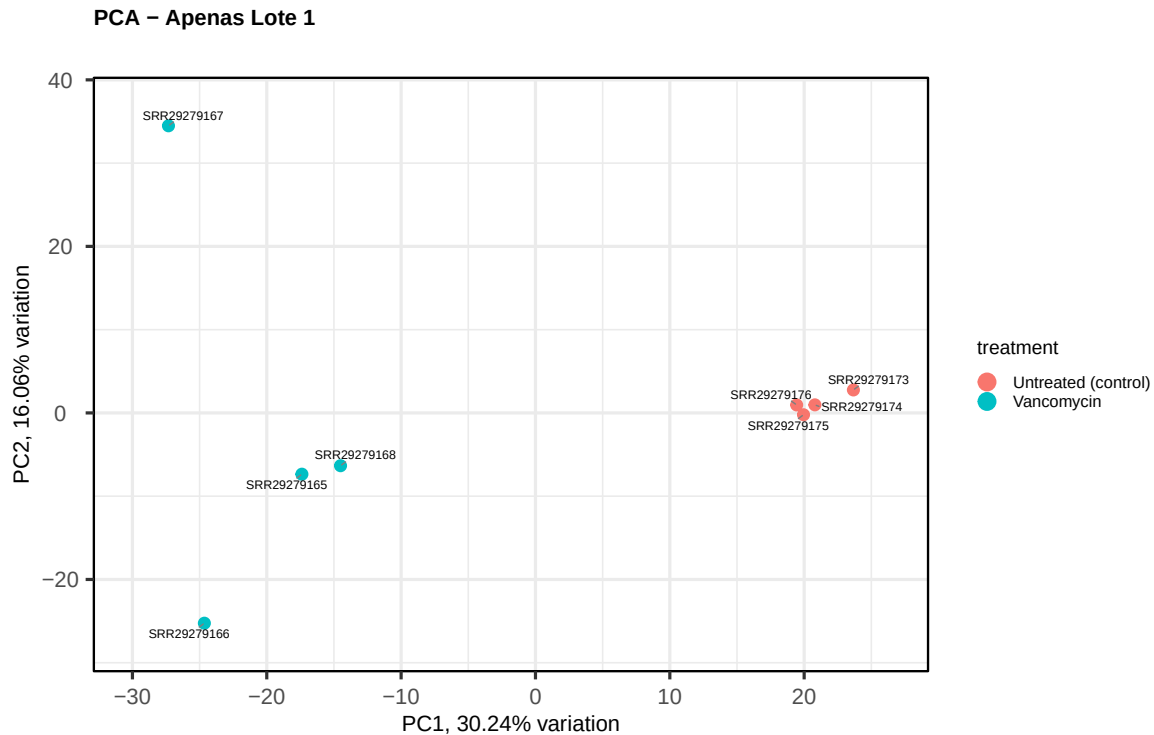
Para ilustrar esse ponto, vamos repetir a PCA usando apenas as amostras do **lote 1** (um par vancomicina/controle com suas réplicas técnicas):

```
# Filtrar apenas amostras do lote 2
meta_batch1 <- metadata[metadata$experiment_group == "1", , drop = FALSE]
counts_batch1 <- counts_vst[, rownames(meta_batch1)]

pca_batch1 <- pca(counts_batch1, metadata = meta_batch1)

biplot(pca_batch1,
       colby = "treatment",
```

```
legendPosition = "right",
title = "PCA - Apenas Lote 1")
```



Note como, isolando um único lote, a separação entre vancomicina e controle se torna mais evidente e as primeiras componentes explicam uma proporção maior da variância. Isso reforça a necessidade de incluir o `experiment_group` como covariável no nosso modelo.

7.1.8 8. Clusterização Hierárquica

A clusterização hierárquica agrupa as amostras com base na **distância euclidiana** entre seus perfis de expressão, construindo um **dendrograma**. Amostras com perfis mais similares são conectadas mais próximas na árvore.

```
# Transpor a matriz: cada amostra vira uma linha

counts_vst_t <- t(counts_vst)
```

```

# Calcular distâncias euclidianas

d <- dist(counts_vst_t, method = "euclidean")

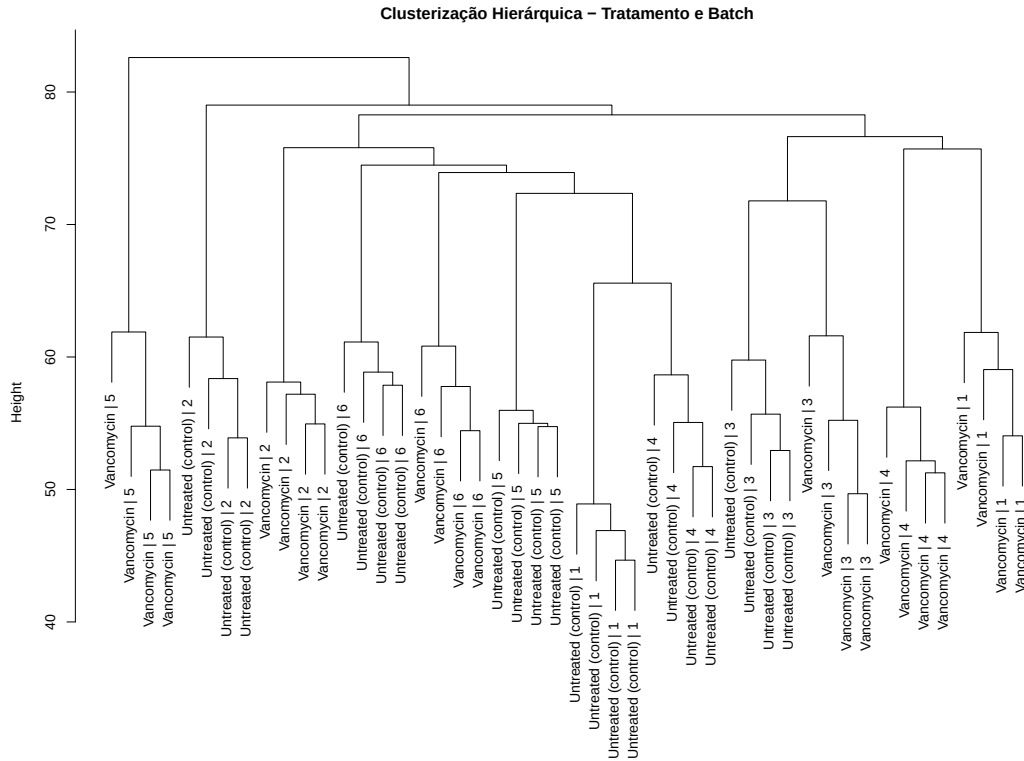
# Clusterizar com ligação completa

clusters <- hclust(d, method = "complete")

# 1. Criar um vetor de novos rótulos combinando as colunas
# Garantimos que a ordem dos metadados seja a mesma das amostras na matriz
novos_rotulos <- paste(metadata[rownames(counts_vst_t), "treatment"],
                        metadata[rownames(counts_vst_t), "experiment_group"],
                        sep = " | ")

# 2. Plotar o dendrograma usando o argumento 'labels'
par(mar = c(4, 4, 2, 8)) # Aumentar a margem direita se os nomes forem longos
plot(clusters,
      labels = novos_rotulos,
      main = "Clusterização Hierárquica - Tratamento e Batch",
      xlab = "",
      sub = "",
      cex = 1) # Font size

```



7.1.8.1 Como interpretar o dendrograma:

- Amostras que compartilham um ramo próximo possuem perfil de expressão semelhante.
- O ideal é que as amostras se agrupem por **tratamento** (vancomicina vs. controle) e não por outras variáveis como o lote experimental.
- Se amostras de grupos diferentes estiverem misturadas, isso pode indicar um efeito biológico fraco ou um forte efeito de *batch*.

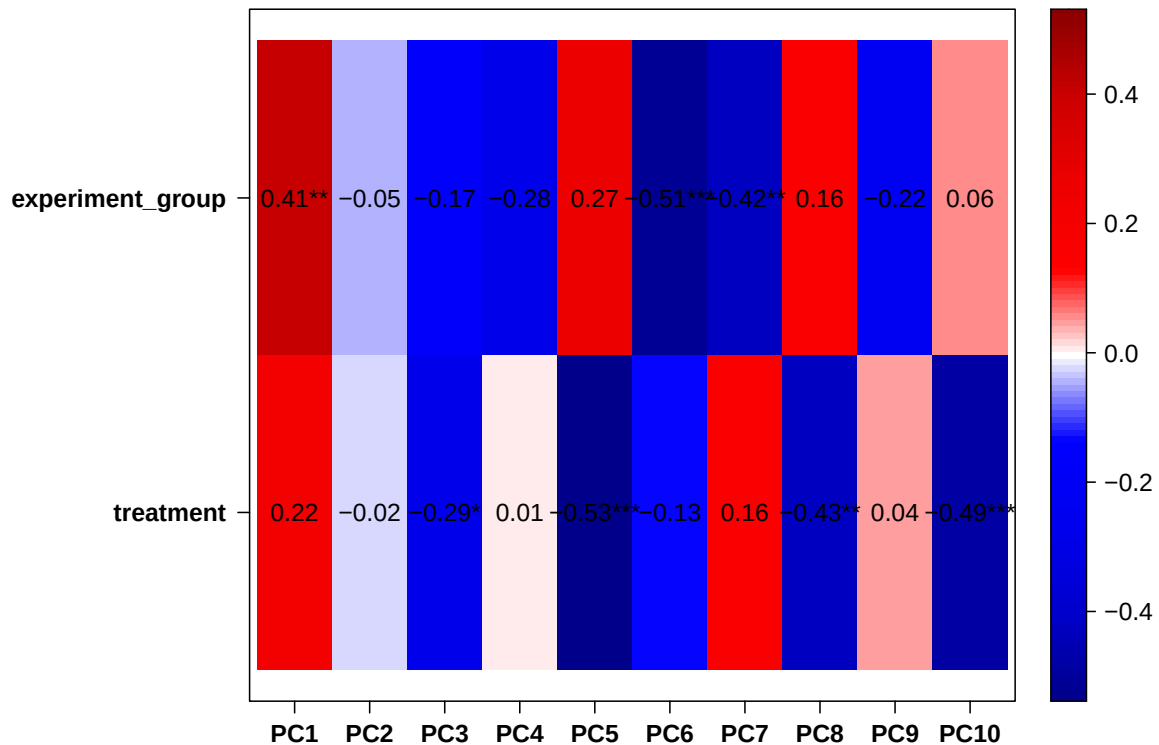
7.1.9 9. Análise de Covariáveis

Uma análise de expressão diferencial é, em essência, **um modelo de regressão** aplicado a cada gene. A variável principal é o tratamento (vancomicina vs. controle), mas outras variáveis, como o grupo experimental (lote/batch), podem influenciar significativamente a expressão.

A função **eigencorplot** nos permite visualizar quais variáveis do metadado se correlacionam com as primeiras componentes principais, ajudando na decisão de **quais covariáveis incluir no modelo**.

```
eigencorplot(pca_result,
             metavars = c("treatment", "experiment_group"),
             main = "Correlação entre Covariáveis e Componentes Principais")
```

Correlação entre Covariáveis e Componentes Principais



! Por que incluir covariáveis?

Se o `experiment_group` (lote biológico) se correlaciona fortemente com alguma das primeiras componentes, isso significa que parte da variabilidade que observamos vem do lote de cultivo celular, e não do efeito da vancomicina. Ao incluir essa variável no modelo do DESeq2 (`~ experiment_group + treatment`), “descontamos” esse efeito e aumentamos o poder estatístico para detectar os genes realmente alterados pelo tratamento.

A sanidade dos nossos dados nos deu uma visão geral da sua estrutura. As amostras

se comportam como esperado? Há outliers que precisam de atenção? Com essas respostas em mãos, podemos avançar com confiança para a próxima etapa.

7.2 Análise de Expressão Diferencial de Genes (DGE)

Agora chegamos à pergunta central deste curso: **quais genes da *Candida albicans* mudaram de comportamento em resposta à vancomicina?**

A análise de Expressão Diferencial de Genes (DGE) é, essencialmente, um modelo de regressão aplicado a cada gene individualmente. O DESeq2 implementa um **Modelo Linear Generalizado (GLM)** baseado na distribuição **Binomial Negativa**, que é particularmente adequada para dados de contagem de RNA-seq, onde a variância geralmente excede a média (sobredispersão).

7.2.1 10. Rodando o DESeq2

A função `DESeq()` congrega três etapas internas:

1. **Estimativa dos fatores de normalização** (`estimateSizeFactors`): corrige diferenças na profundidade de sequenciamento entre amostras.
2. **Estimativa da dispersão** (`estimateDispersions`): modela a variabilidade dos dados gene a gene, compartilhando informação entre genes com expressão similar.
3. **Teste estatístico** (`nbinomWaldTest`): aplica o modelo e testa a significância para cada gene.

Lembre-se que, na seção 5, definimos o modelo do DESeq2 com a fórmula `~ experiment_group + treatment`. É nessa fórmula que resolvemos o problema dos lotes experimentais. A lógica funciona assim:

- A variável `experiment_group` entra como **covariável** (o primeiro termo da fórmula). Isso instrui o DESeq2 a “descontar” a variabilidade que vem das diferenças entre os 6 lotes **antes** de avaliar o efeito do tratamento.
- A variável `treatment` é a **variável de interesse** (o último termo). O teste estatístico será aplicado sobre ela.

Na prática, o modelo estima para cada gene: “*quanto da expressão se explica pelo lote?*” e, removendo esse efeito, pergunta: “*o que sobra é explicado pelo tratamento com vancomicina?*”. Sem essa correção, diferenças entre lotes poderiam ser falsamente atribuídas ao tratamento — ou, inversamente, mascarar efeitos reais.

```
# Rodar o pipeline completo do DESeq2
dds <- DESeq(dds)
```

7.2.2 11. Extrair os Resultados

Precisamos agora extrair os resultados para o **contraste** que nos interessa. Um contraste é a comparação entre dois grupos de uma variável categórica. No nosso caso, queremos comparar **Vancomycin vs. Untreated (control)**.

A função `results()` aceita o argumento `contrast`, um vetor de 3 elementos:

1. O nome da variável (`treatment`).
2. O grupo correspondente ao **tratamento** (`Vancomycin`).
3. O grupo correspondente à **referência** (`Untreated (control)`).

```
res_dge <- DESeq2::results(dds,
                           contrast = c("treatment", "Vancomycin", "Untreated
                                         (control)"))
```

! A Ordem do Contraste Importa!

A interpretação do resultado depende diretamente da ordem com que o contraste é construído. No nosso caso, a referência é o grupo **controle** (não tratado). Portanto:

- **Log2FC positivo** → gene com expressão **maior** nas amostras tratadas com vancomicina (upregulado).
- **Log2FC negativo** → gene com expressão **menor** nas amostras tratadas com vancomicina (downregulado).

Se invertêssemos a ordem do contraste, a interpretação seria a oposta.

Vamos transformar o resultado em um data frame e inspecionar as colunas:

```
res_dge <- as.data.frame(res_dge)
head(res_dge)
```

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	padj				
C1_00010W_A	0.13073479	0.2439030	2.5493706	0.09567186	0.9237812
NA					

C1_00020C_A	1.35871278	-0.7858614	0.5442897	-1.44382928	0.1487870
	0.5357854				
C1_00030C_A	0.02292633	0.2485875	2.9448570	0.08441413	0.9327272
	NA				
C1_00040W_A	0.03442856	0.3688108	2.9448570	0.12523896	0.9003344
	NA				
C1_00050C_A	0.03644129	-0.1120823	2.9448570	-0.03806036	0.9696396
	NA				
C1_00060W_A	0.36778762	0.4121354	1.3458047	0.30623715	0.7594241
	NA				

O data frame `res_dge` contém as seguintes colunas para cada gene:

- **baseMean**: média dos valores de contagens normalizados para todas as amostras.
- **log2FoldChange**: métrica que indica a intensidade e a direção da mudança de expressão (positivo = up, negativo = down).
- **lfcSE**: erro padrão do log2FC.
- **stat**: estatística do teste de Wald.
- **pvalue**: p-valor do teste.
- **padj**: p-valor ajustado pelo método de Benjamini-Hochberg (FDR), que corrige para os múltiplos testes (um por gene).

Ponto Importante

O DESeq2 atribui NA ao `padj` de genes com contagens muito baixas ou com dispersão extrema. Esses genes não possuem evidência estatística suficiente para serem testados e, na prática, são considerados não significativos.

7.2.3 12. Definindo os Genes Diferencialmente Expressos

Para identificar os genes significativos, vamos utilizar o critério de **padj 0.1**. Além disso, vamos classificar cada gene como *Upregulated*, *Downregulated* de acordo com o log2FC ou *Not significant*:

Nota Didática

Em publicações científicas, o corte mais comumente utilizado é **padj 0.05**. Neste mini-curso, utilizamos um limiar mais permissivo (**padj 0.1**) para fins didáticos, permitindo que tenhamos um número maior de genes diferencialmente expressos para demonstrar as análises downstream (enriquecimento funcional e visualizações). Em um cenário de pesquisa real, recomendamos utilizar `padj 0.05`.

```

res_dge <- res_dge %>%
  mutate(
    padj = ifelse(is.na(padj), 1, padj),
    signif = case_when(
      padj <= 0.1 & log2FoldChange > 0 ~ "Upregulated",
      padj <= 0.1 & log2FoldChange < 0 ~ "Downregulated",
      padj > 0.1 ~ "Not significant"
    )
  )

# Resumo dos genes significativos
table(res_dge$signif)

```

Downregulated	Not significant	Upregulated
35	6393	40

7.2.4 13. MA Plot

O **MA Plot** é a primeira visualização diagnóstica da expressão diferencial. Ele relaciona a **expressão média** de cada gene (eixo X, em escala log) com o **log2 Fold Change** estimado (eixo Y). Genes significativos aparecem em destaque.

```

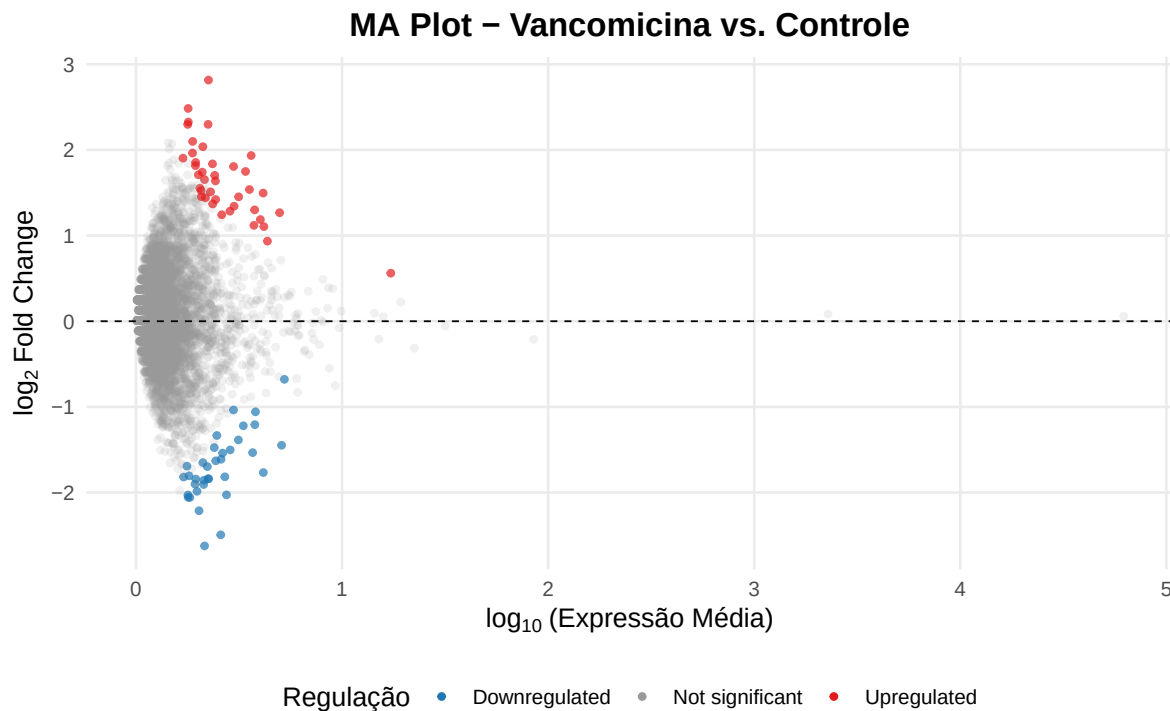
ggplot(res_dge, aes(x = log10(baseMean + 1), y = log2FoldChange, color =
  ↪ signif)) +
  geom_point(aes(alpha = signif), size = 1.2) +
  scale_color_manual(
    values = c("Downregulated" = "#1F78B4",
              "Not significant" = "grey60",
              "Upregulated" = "#E31A1C"),
    name = "Regulação"
  ) +
  scale_alpha_manual(
    values = c("Downregulated" = 0.7,
              "Not significant" = 0.15,
              "Upregulated" = 0.7),
    guide = "none"
  ) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "black", linewidth
  ↪ = 0.4) +
  labs(

```

```

title = "MA Plot - Vancomicina vs. Controle",
x = expression(log[10] ~ "(Expressão Média)"),
y = expression(log[2] ~ "Fold Change")
) +
theme_minimal(base_size = 13) +
theme(
  plot.title = element_text(hjust = 0.5, face = "bold"),
  legend.position = "bottom",
  panel.grid.minor = element_blank()
)

```



7.2.4.1 Como interpretar o MA Plot:

- Genes à **esquerda** (baixo baseMean) possuem baixa expressão. Mudanças grandes nesse intervalo devem ser interpretadas com cautela, pois a incerteza estatística é alta.
- Genes **significativos** (coloridos) que se afastam da linha central ($\log_2FC = 0$) são os que realmente mudaram de expressão entre as condições.
- O ideal é que a nuvem de pontos cinza esteja **centralizada em zero**, indicando que a normalização foi adequada.

7.2.5 14. Volcano Plot

O **Volcano Plot** é o gráfico mais utilizado para visualizar resultados de expressão diferencial. Ele combina a **significância estatística** ($-\log_{10}$ do p-valor ajustado, eixo Y) com a **magnitude da mudança** ($\log_2\text{FC}$, eixo X), permitindo identificar rapidamente os genes mais relevantes.

```
library(ggrepel)

padj_cutoff <- 0.1 # Limiar didático (em pesquisa, use 0.05)

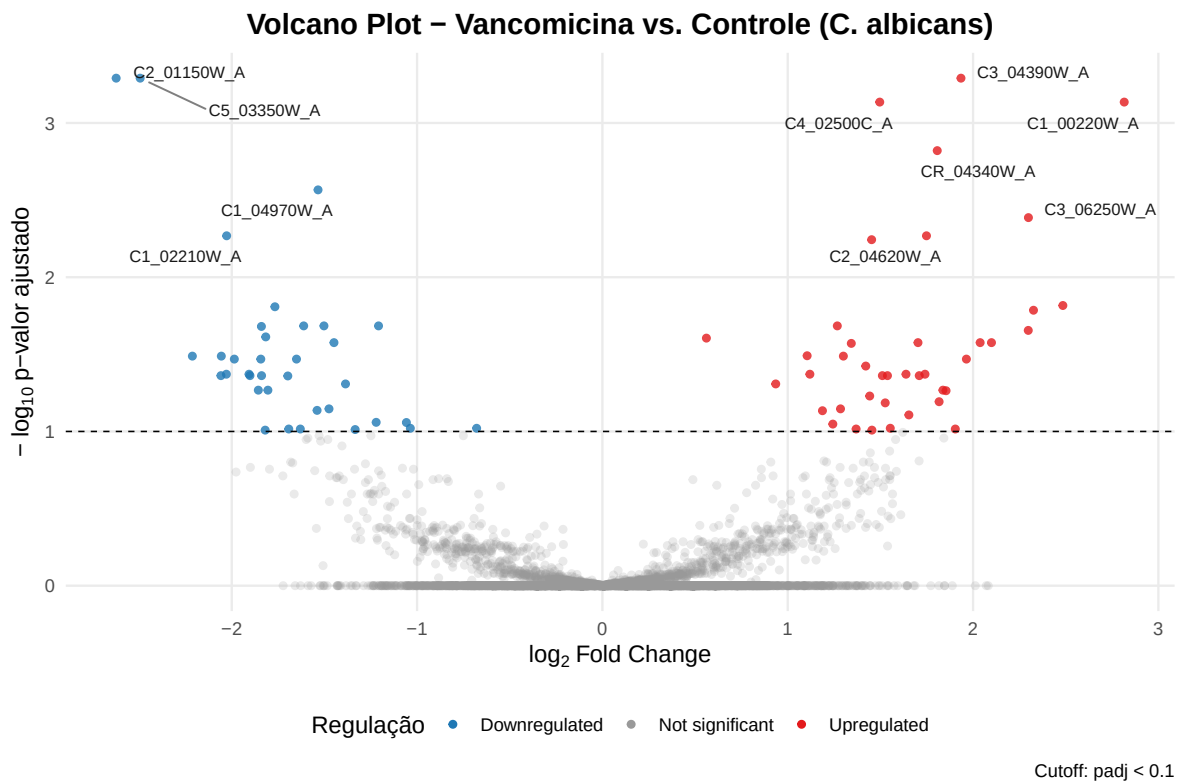
# Selecionar os top 10 genes mais significativos para rotular
genes_to_label <- res_dge %>%
  filter(signif != "Not significant") %>%
  arrange(padj) %>%
  head(10)

ggplot(res_dge, aes(x = log2FoldChange, y = -log10(padj), color = signif)) +
  geom_point(aes(alpha = signif), size = 1.5) +
  scale_color_manual(
    values = c("Downregulated" = "#1F78B4",
               "Not significant" = "grey60",
               "Upregulated" = "#E31A1C"),
    name = "Regulação"
  ) +
  scale_alpha_manual(
    values = c("Downregulated" = 0.8,
               "Not significant" = 0.2,
               "Upregulated" = 0.8),
    guide = "none"
  ) +
  geom_hline(yintercept = -log10(padj_cutoff),
             linetype = "dashed", color = "black", linewidth = 0.4) +
  geom_text_repel(
    data = genes_to_label,
    aes(label = rownames(genes_to_label)),
    size = 3,
    box.padding = 0.5,
    point.padding = 0.3,
    max.overlaps = Inf,
    segment.color = "grey50",
    color = "gray9"
  ) +
  labs(
```

```

title = "Volcano Plot - Vancomicina vs. Controle (C. albicans)",
x = expression(log[2] ~ "Fold Change"),
y = expression("-" ~ log[10] ~ "p-valor ajustado"),
caption = paste0("Cutoff: padj < ", padj_cutoff)
) +
theme_minimal(base_size = 13) +
theme(
  plot.title = element_text(hjust = 0.5, face = "bold"),
  legend.position = "bottom",
  panel.grid.minor = element_blank(),
  panel.background = element_rect(fill = "white", color = NA),
  plot.background = element_rect(fill = "white", color = NA)
)

```



7.2.5.1 Como interpretar o Volcano Plot:

- **Eixo X ($\log_2\text{FC}$):** genes à direita estão upregulados sob o tratamento de vancomicina; genes à esquerda estão downregulados.

- **Eixo Y (-log10 padj):** quanto mais alto o ponto, mais significativa é a mudança. A linha tracejada marca o threshold de $\text{padj} = 0.1$.
- Os **genes rotulados** são os top 10 mais significativos.

💡 O que o Volcano Plot nos diz biologicamente?

Observe a distribuição entre upregulados e downregulados. Se o tratamento com vancomicina remodela a resposta transcricional da *Candida*, esperamos ver genes envolvidos em **resistência a estresse**, **remodelamento da parede celular** e **virulência** entre os mais significativos. Genes de *housekeeping* devem permanecer na região cinza central.

7.2.6 15. Heatmap dos Genes Diferencialmente Expressos

O **heatmap** nos permite visualizar o padrão de expressão dos genes significativos em todas as amostras simultaneamente. Para isso, utilizamos os valores de expressão normalizados transformados em **z-score** (escala por gene), o que evidencia as diferenças relativas entre condições.

```
# Filtrar apenas os genes significativos (padj <= 0.05)
res_dge_signif <- subset(res_dge, padj <= 0.05)

# Obter contagens normalizadas
exp_genes <- DESeq2::counts(dds, normalized = TRUE)

# Selecionar apenas os genes DEG
idx <- rownames(exp_genes) %in% rownames(res_dge_signif)
exp_dge <- exp_genes[idx, ]

# Transformar em z-score (por gene)
exp_dge_z <- t(apply(exp_dge, 1, scale, center = TRUE, scale = TRUE))
colnames(exp_dge_z) <- colnames(exp_dge)

# Criar anotação das colunas
ann_col <- data.frame(
  Tratamento = metadata$treatment,
  Lote = metadata$experiment_group,
  row.names = rownames(metadata)
)

ann_colors <- list(
  Tratamento = c("Untreated (control)" = "#4393C3", "Vancomycin" =
    ↪ "#D6604D"),
```

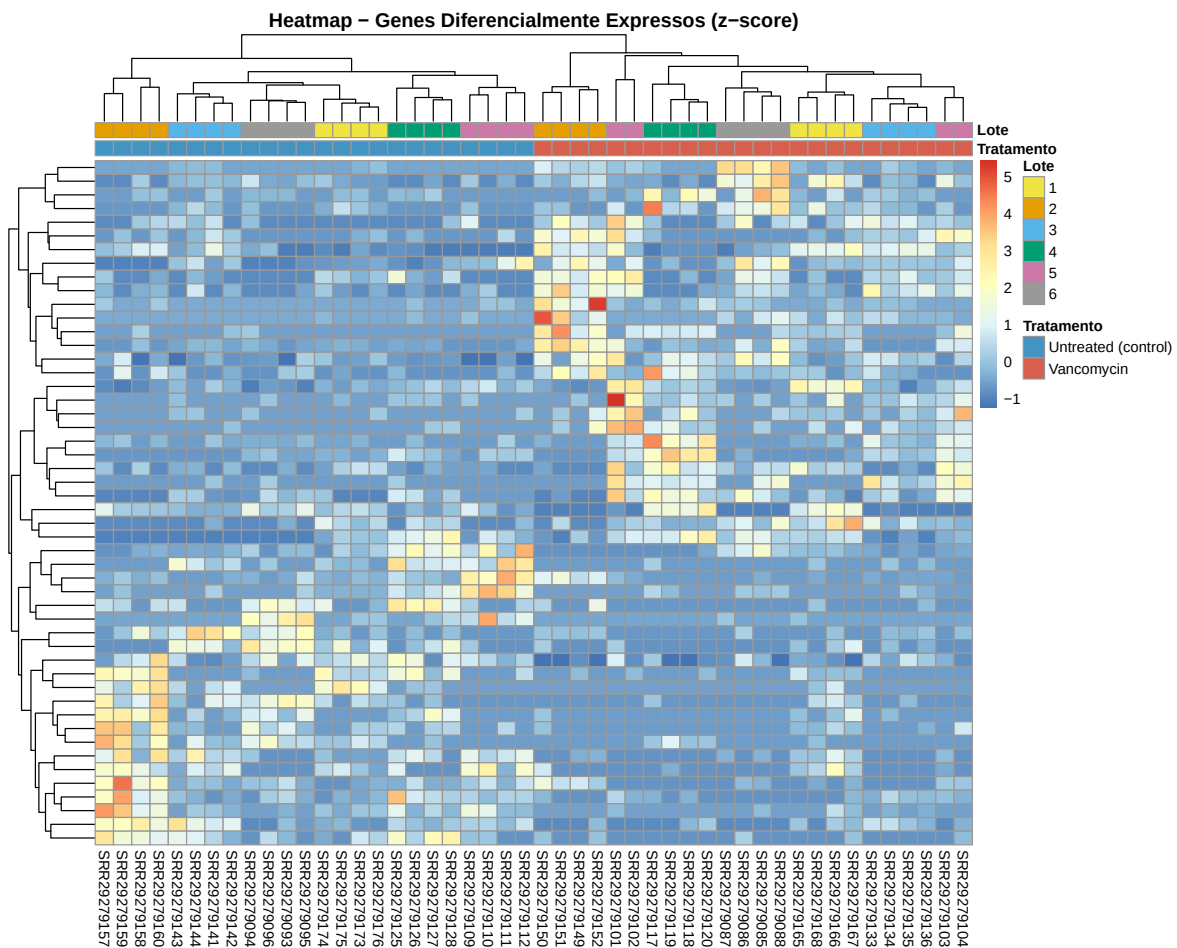


```

Lote = c("1" = "#F0E442", "2" = "#E69F00", "3" = "#56B4E9",
        "4" = "#009E73", "5" = "#CC79A7", "6" = "#999999")
)

# Plotar heatmap
pheatmap(exp_dge_z,
  show_rownames = FALSE,
  annotation_col = ann_col,
  annotation_colors = ann_colors,
  cluster_cols = TRUE,
  fontsize = 9,
  main = "Heatmap - Genes Diferencialmente Expressos (z-score)")

```



7.2.6.1 Como interpretar o heatmap:

- Cada **linha** é um gene diferencialmente expresso e cada **coluna** é uma amostra.
- As **cores** representam o z-score: **vermelho** indica expressão acima da média daquele gene, enquanto **azul** indica expressão abaixo da média.
- A **barra de anotação superior** diferencia as amostras por tratamento e lote.
- Se os genes DEG segregam perfeitamente as amostras por tratamento, isso confirma que o sinal biológico é forte e consistente entre os lotes.
- Note que as **linhas são clusterizadas**: genes com padrões de expressão similares ficam próximos, revelando blocos de genes co-regulados.

```
# Salvar a tabela de genes diferencialmente expressos para a próxima etapa
saveRDS(res_dge, file = here::here("data/DGE/res_dge.rds"))
```

Identificamos os genes da *Candida albicans* cuja expressão mudou significativamente em resposta à vancomicina. Mas o que esses genes fazem? A quais vias biológicas eles pertencem? Essas perguntas serão respondidas na próxima seção, onde realizaremos a análise de enriquecimento funcional.

8 Análises Downstream

Na etapa anterior, identificamos quais genes da *Candida albicans* tiveram sua expressão significativamente alterada pelo tratamento com vancomicina. Mas uma lista de genes, por si só, diz pouco sobre a biologia do organismo. Para extrairmos significado biológico, precisamos perguntar: **esses genes pertencem a vias biológicas específicas? Há processos celulares mais afetados do que o esperado ao acaso?**

É exatamente isso que a **análise de enriquecimento funcional** nos permite responder.

9 Enriquecimento Funcional

A análise de enriquecimento funcional verifica se, entre os genes diferencialmente expressos, há um número desproporcional de genes pertencentes a uma determinada via biológica ou processo celular. Se encontrarmos mais genes alterados em uma via do que o esperado por acaso, dizemos que essa via está **enriquecida**.

A abordagem que utilizaremos é a **ORA (Over-Representation Analysis)**, que parte da nossa lista de genes significativos e aplica um teste hipergeométrico para verificar quais vias estão sobre-representadas. Para os bancos de dados, utilizaremos o **KEGG Pathways** e o **Gene Ontology (GO)**.

9.1 O que são GO e KEGG?

9.1.1 Gene Ontology (GO)

O [Gene Ontology](#) é um sistema de ontologias que descreve a função dos genes em três dimensões independentes (ontologias):

- **Biological Process (BP)**: o processo celular de que o gene participa (ex: *resposta a estresse oxidativo, biossíntese da parede celular*).
- **Molecular Function (MF)**: a atividade molecular que a proteína exerce (ex: *atividade de quinase, ligação a ATP*).
- **Cellular Component (CC)**: onde no espaço celular a proteína atua (ex: *núcleo, membrana plasmática*).

A GO é organizada como um **grafo acíclico dirigido (DAG)**: termos mais gerais (ex: *processo metabólico*) estão no topo da hierarquia, e termos mais específicos (ex: *biossíntese de ergosterol*) estão na base. Um gene pode ser anotado em múltiplos termos e em múltiplas ontologias simultaneamente.

9.1.2 KEGG Pathways

O KEGG ([Kyoto Encyclopedia of Genes and Genomes](#)) é um banco de dados que organiza o conhecimento biológico em **mapas de vias metabólicas e de sinalização**. Cada via é um diagrama que conecta enzimas, metabólitos e reações em sequência, representando processos como a glicólise, o ciclo do TCA ou a biossíntese de aminoácidos.

Diferente do GO, o KEGG tem uma estrutura **plana** (não hierárquica): cada via é independente e autocontida. Além disso, o KEGG inclui informações sobre **reações bioquímicas e compostos químicos**, o que o torna especialmente útil para estudar metabolismo.

9.1.3 Principais diferenças

Característica	Gene Ontology (GO)	KEGG Pathways
Estrutura	Hierárquica (DAG)	Plana (mapas independentes)
Foco	Funções, processos e localização	Vias metabólicas e de sinalização
Granularidade	Alta (termos muito específicos a gerais)	Moderada (vias completas)
Curadoria	Comunidade científica global	Equipe centralizada (Kanehisa Lab)
Organismo	Ampla cobertura (via OrgDb ou GAF)	Código por organismo (ex: cal)

9.2 A Importância dos Bancos de Dados Curados

Antes de começar, é fundamental entender que **o enriquecimento funcional depende inteiramente da qualidade e completude das anotações disponíveis para o organismo estudado**. Para organismos-modelo como *Homo sapiens* ou *Mus musculus*, bancos como KEGG, Gene Ontology e Reactome possuem anotações extensas, curadas por décadas de pesquisa. O resultado é que milhares de genes estão anotados em centenas de vias biológicas, e o teste estatístico tem alta chance de encontrar padrões significativos.

Para organismos não-modelo como *Candida albicans*, a situação é diferente:

- **Menos genes anotados:** uma fração menor do genoma possui anotação funcional.
- **Menos vias catalogadas:** o número de vias biológicas mapeadas é mais restrito.
- **Anotações menos curadas:** muitas anotações são inferidas computacionalmente (IEA - *Inferred from Electronic Annotation*), e não por evidência experimental direta.

Isso significa que, mesmo que nossos genes estejam participando de processos biológicos relevantes, **se esses processos não estiverem adequadamente anotados no banco de dados, o enriquecimento simplesmente não os detectará**. Por isso, para *C. albicans*, utilizaremos tanto o KEGG (que suporta o organismo nativamente) quanto as anotações curadas do **Candida Genome Database (CGD)**, o banco de referência para genômica de *Candida*, que possui mais de 33 mil anotações GO específicas.

⚠ Nota Didática — Thresholds Relaxados

Em publicações científicas, os cortes padrão para enriquecimento são **p.adjust 0.05** e **qvalue 0.05**. Neste minicurso, utilizamos thresholds mais permissivos (**p.adjust 0.5** e **qvalue 0.5**) para fins **didáticos**, de modo que seja possível visualizar e interpretar os gráficos de enriquecimento mesmo com um número reduzido de DEGs. Em um cenário de pesquisa real, recomendamos utilizar os cortes padrão.

9.3 1. Carregando os Pacotes

```
suppressPackageStartupMessages({
  library(clusterProfiler)
  library(KEGGREST)
  library(enrichplot)
  library(GO.db)
  library(dplyr)
  library(readr)
  library(tibble)
  library(ggplot2)
})
```

9.4 2. Carregando os Resultados da Expressão Diferencial

Vamos recuperar a tabela de resultados que salvamos no módulo anterior:

```
res_dge <- readRDS(here::here("data/DGE/res_dge.rds"))
head(res_dge)
```

baseMean	log2FoldChange	lfcSE	stat	pvalue
padj				

C1_00010W_A	0.13073479	0.2439030	2.5493706	0.09567186	0.9237812
C1_00020C_A	1.35871278	-0.7858614	0.5442897	-1.44382928	0.1487870
C1_00030C_A	0.02292633	0.2485875	2.9448570	0.08441413	0.9327272
C1_00040W_A	0.03442856	0.3688108	2.9448570	0.12523896	0.9003344
C1_00050C_A	0.03644129	-0.1120823	2.9448570	-0.03806036	0.9696396
C1_00060W_A	0.36778762	0.4121354	1.3458047	0.30623715	0.7594241

	signif
C1_00010W_A	Not significant
C1_00020C_A	Not significant
C1_00030C_A	Not significant
C1_00040W_A	Not significant
C1_00050C_A	Not significant
C1_00060W_A	Not significant

9.5 3. Preparando os Conjuntos de Genes

Vamos preparar três conjuntos de genes para o enriquecimento: os **upregulados**, os **down-regulados** e **todos os DEGs** juntos.

```
genes_up <- rownames(subset(res_dge, signif == "Upregulated"))
genes_down <- rownames(subset(res_dge, signif == "Downregulated"))
genes_all <- rownames(subset(res_dge, signif != "Not significant"))

cat("Genes upregulados:", length(genes_up), "\n")
```

Genes upregulados: 40

```
cat("Genes downregulados:", length(genes_down), "\n")
```

Genes downregulados: 35

```
cat("Total DEGs:", length(genes_all), "\n")
```

Total DEGs: 75

9.6 4. Conversão de Identificadores para o KEGG

Os nossos identificadores de genes seguem o padrão do Ensembl Fungi (ex: C1_00010W_A), mas o KEGG utiliza um formato próprio para *C. albicans* (ex: CAALFM_C100010WA). Precisamos construir o dicionário para conversão:

```
# Baixar os nomes dos genes de C. albicans (cal) do KEGG
kegg_genes <- keggList("cal")

# Criar tabela de mapeamento
kegg_mapping <- data.frame(
  kegg_id = sub("cal:", "", names(kegg_genes)),
  description = as.character(kegg_genes),
  stringsAsFactors = FALSE
)

# Gerar o ID Ensembl Fungi equivalente a partir do KEGG ID
kegg_mapping <- kegg_mapping %>%
  mutate(
    core = sub("^CAALFM_", "", kegg_id),
    ensembl_id = gsub(
      "(C[0-9R]+)(\\d{5})(\\w)(\\w)$",
      "\\1_\\2\\3_\\4",
      core
    )
  )

# Verificar quantos dos nossos genes têm correspondência
genes_com_kegg <- sum(rownames(res_dge) %in% kegg_mapping$ensembl_id)
cat("Genes mapeados para KEGG:", genes_com_kegg, "de", nrow(res_dge), "\n")
```

Genes mapeados para KEGG: 6117 de 6468

```
# Criar dicionário: Ensembl → KEGG
ensembl_to_kegg <- setNames(kegg_mapping$kegg_id, kegg_mapping$ensembl_id)
```

Esse dicionário será usado adiante tanto para o enriquecimento KEGG via GSEA (seção 5) quanto para a construção da rede PPI com o STRING (seção 11).

9.7 5. Enriquecimento KEGG — GSEA

Diferente do **ORA**, que parte de uma lista discreta de DEGs (filtrada por um cutoff de significância), o **GSEA (Gene Set Enrichment Analysis)** considera **todos os genes** ranqueados por uma métrica contínua — tipicamente o **log2FoldChange** — e verifica se os membros de cada via biológica se concentram nas extremidades do ranking. A pergunta muda:

- **ORA**: “Há mais DEGs nesta via do que o esperado ao acaso?”
- **GSEA**: “Os genes desta via estão sistematicamente entre os mais up- ou down-regulados?”

A vantagem é que o GSEA **não precisa de cutoff**: ele aproveita o sinal coordenado de muitos genes com mudanças pequenas, que o ORA descartaria. O resultado vem na forma de um **Normalized Enrichment Score (NES)**: positivo indica que a via está enriquecida na ponta UP do ranking, negativo na ponta DOWN.

9.7.1 Construindo o vetor ranqueado

Para o GSEA, precisamos transformar a tabela `res_dge` em um **vetor numérico nomeado** com todos os genes ranqueados por `log2FC` e identificados pelo formato KEGG (`CAALFM_`). Vamos usar o dicionário `ensembl_to_kegg` que construímos na seção 4:

```
suppressPackageStartupMessages(library(stringr))

# Vetor: log2FC com nome = Ensembl ID
ranks <- res_dge$log2FoldChange
names(ranks) <- rownames(res_dge)
ranks <- ranks[!is.na(ranks)]

# Converter os nomes para IDs KEGG e ordenar decrescentemente
ranks_kegg <- ranks[names(ranks) %in% names(ensembl_to_kegg)]
names(ranks_kegg) <- ensembl_to_kegg[names(ranks_kegg)]
ranks_kegg <- sort(ranks_kegg[!duplicated(names(ranks_kegg))], decreasing =
  ↪ TRUE)

cat("Genes no ranking (KEGG IDs únicos):", length(ranks_kegg), "\n")
```

```
Genes no ranking (KEGG IDs únicos): 6023
```

```
cat("log2FC - máximo (topo do ranking) :", round(max(ranks_kegg), 2), "\n")
```

```
log2FC - máximo (topo do ranking) : 2.82
```

```
cat("log2FC - mínimo (base do ranking) :", round(min(ranks_kegg), 2), "\n")
```

```
log2FC - mínimo (base do ranking) : -2.62
```

9.7.2 Rodando gseKEGG()

A função `gseKEGG()` consulta a API REST do KEGG em tempo real para obter as anotações de vias da *C. albicans* (`organism = "cal"`) e roda o algoritmo de permutação. Como o GSEA é estocástico, fixamos uma semente com `set.seed()` para garantir que duas execuções da mesma versão do banco produzam exatamente o mesmo resultado:

```
set.seed(42) # GSEA é estocástica (usa permutações); set.seed garante
↳ reprodutibilidade
gsea_kegg <- gseKEGG(
  geneList      = ranks_kegg,
  organism      = "cal",
  keyType       = "kegg",
  minGSSize     = 5,
  maxGSSize     = 500,
  pvalueCutoff  = 0.5,
  pAdjustMethod = "BH",
  eps           = 0,
  verbose       = FALSE
)

cat("Vias enriquecidas (GSEA):", nrow(as.data.frame(gsea_kegg)), "\n")
```

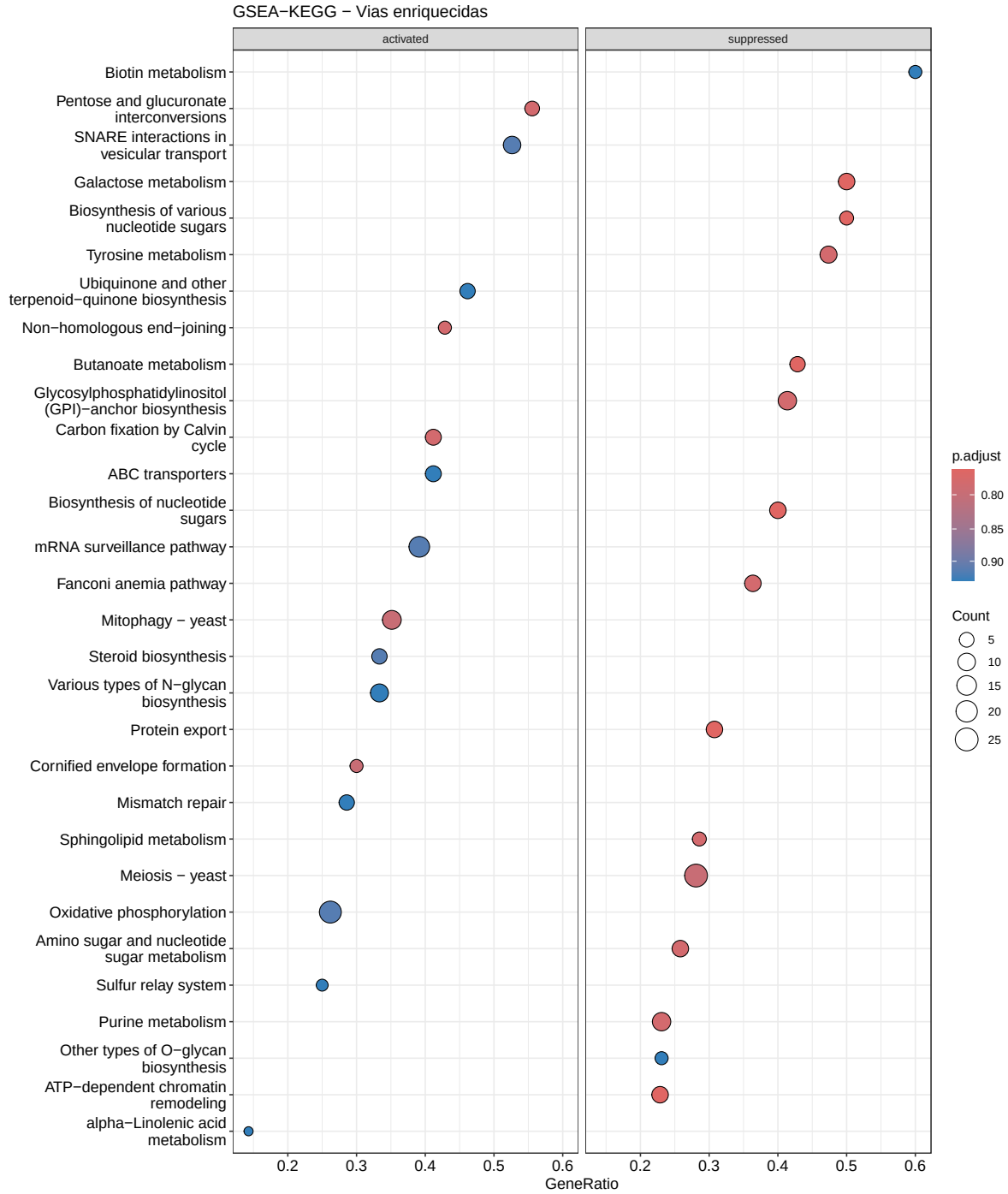
```
Vias enriquecidas (GSEA): 62
```

9.7.3 Dotplot — GSEA KEGG (UP vs. DOWN)

Diferente do dotplot do ORA, o do GSEA já vem separado por direção: cada via aparece no painel **activated** ($NES > 0$, regulada para cima) ou **suppressed** ($NES < 0$, regulada para baixo).

```
if (!is.null(gsea_kegg) && nrow(as.data.frame(gsea_kegg)) > 0) {
  dotplot(gsea_kegg,
    showCategory = 15,
    split = ".sign",
    title = "GSEA-KEGG - Vias enriquecidas") +
```

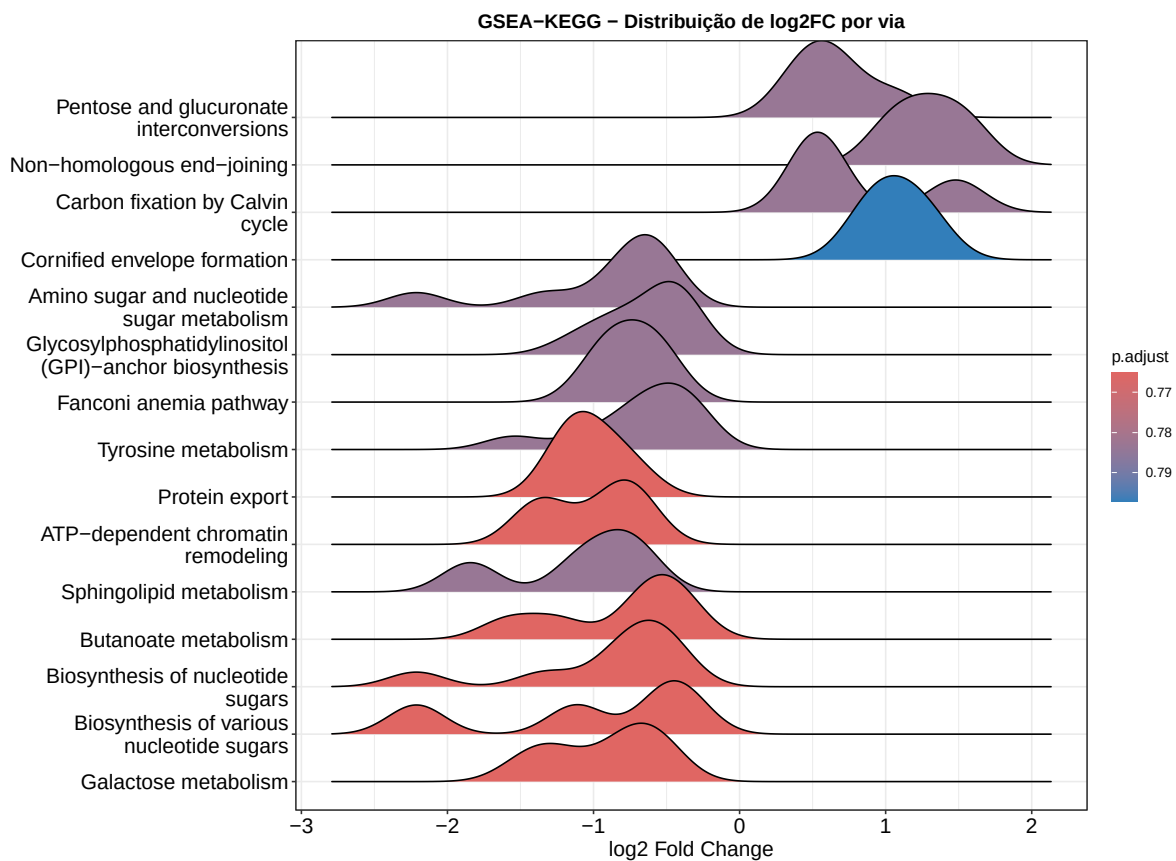
```
    facet_grid(. ~ .sign) +  
    scale_color_gradient(low = "#E31A1C", high = "#FCBBA1", name =  
      ↪ "p-valor\najustado")  
  } else {  
    cat("Nenhuma via KEGG enriquecida.\n")  
  }
```



9.7.4 Ridgeplot — distribuição de log2FC por via

O ridgeplot mostra, para cada via enriquecida, a distribuição dos log2FC dos seus genes membros. Vias “puxadas” para a direita estão dominantes em UP; para a esquerda, em DOWN.

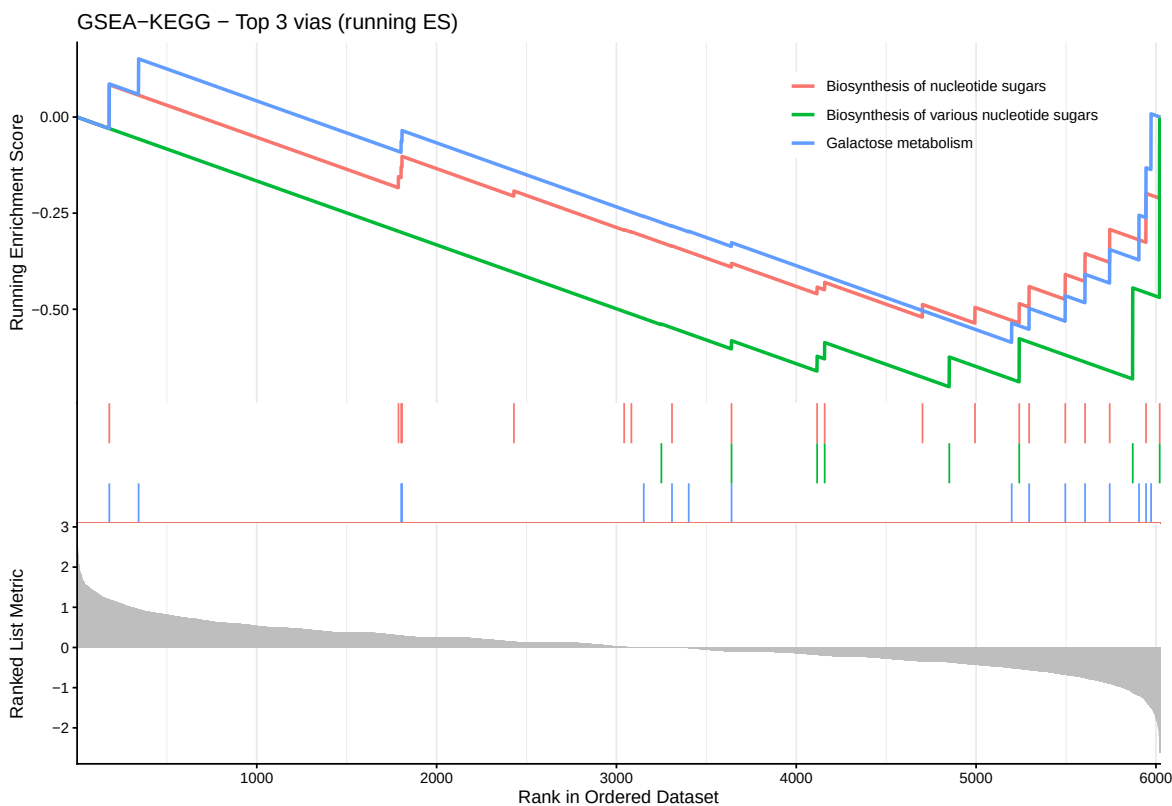
```
if (!is.null(gsea_kegg) && nrow(as.data.frame(gsea_kegg)) > 0) {  
  ridgeplot(gsea_kegg, showCategory = 15) +  
    labs(title = "GSEA-KEGG - Distribuição de log2FC por via",  
         x = "log2 Fold Change") +  
    theme(plot.title = element_text(face = "bold", hjust = 0.5))  
} else {  
  cat("Nenhuma via KEGG enriquecida.\n")  
}
```



9.7.5 Curva de Running Enrichment Score — Top 3 vias

A curva clássica do GSEA mostra como o Enrichment Score “cresce” ou “decrece” à medida que percorremos o ranking do topo (mais UP) até a base (mais DOWN). Picos positivos no início indicam vias UP; vales negativos no final indicam vias DOWN. As marcas verticais (|) abaixo da curva indicam onde caem, no ranking, os genes da via:

```
if (!is.null(gsea_kegg) && nrow(as.data.frame(gsea_kegg)) > 0) {  
  top3 <- as.data.frame(gsea_kegg) %>%  
    arrange(p.adjust) %>%  
    head(3) %>%  
    pull(ID)  
  gseaplot2(gsea_kegg, geneSetID = top3,  
            title = "GSEA-KEGG - Top 3 vias (running ES)")  
} else {  
  cat("Nenhuma via KEGG enriquecida.\n")  
}
```

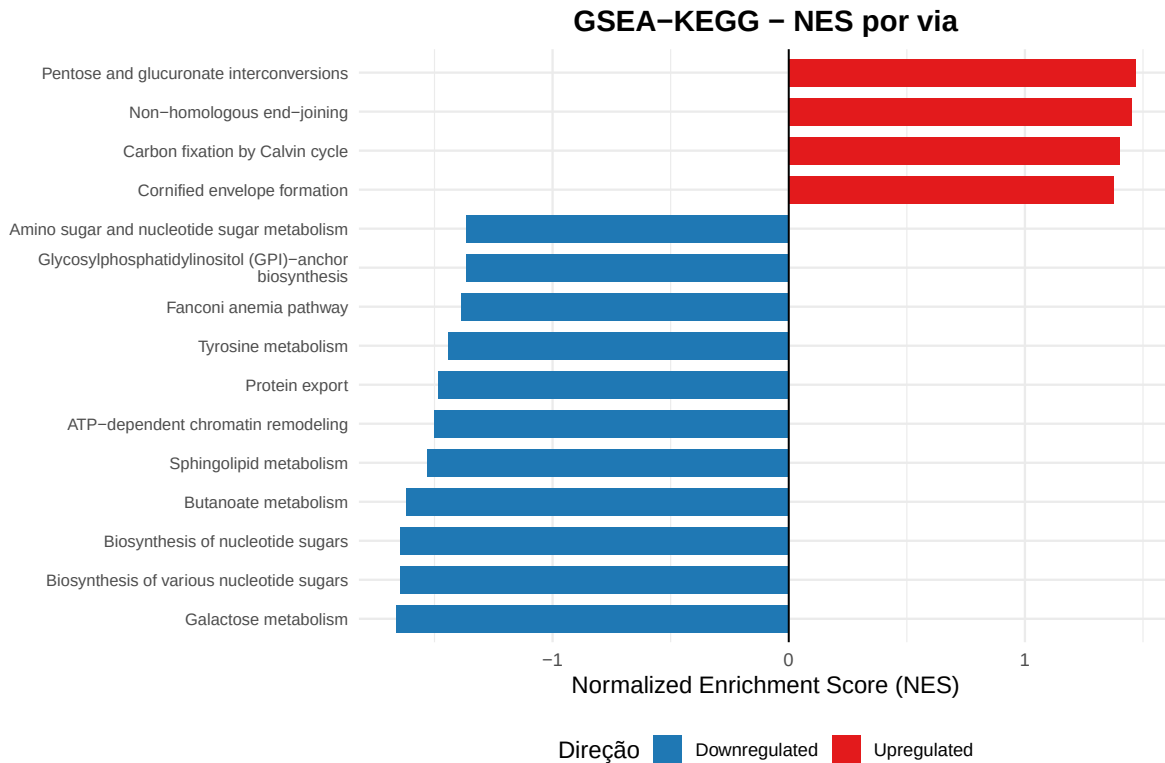


9.7.6 Barplot Bidirecional — NES por via

O análogo direto do barplot bidirecional do ORA é usar o **NES** como métrica: barras vermelhas (UP, NES > 0) crescem para a direita; azuis (DOWN, NES < 0) crescem para a esquerda, compartilhando o mesmo eixo Y:

```
if (!is.null(gsea_kegg) && nrow(as.data.frame(gsea_kegg)) > 0) {
  df_gsea <- as.data.frame(gsea_kegg) %>%
    mutate(Direction = ifelse(NES > 0, "Upregulated", "Downregulated"),
           term_lbl = str_wrap(Description, 45)) %>%
    arrange(desc(abs(NES))) %>%
    head(15)

  ggplot(df_gsea, aes(x = NES, y = reorder(term_lbl, NES), fill = Direction))
  ↪ +
  geom_col(width = 0.7) +
  scale_fill_manual(values = c("Upregulated" = "#E31A1C",
                              "Downregulated" = "#1F78B4")) +
  geom_vline(xintercept = 0, linewidth = 0.5) +
  labs(title = "GSEA-KEGG - NES por via",
       x = "Normalized Enrichment Score (NES)",
       y = NULL, fill = "Direção") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        axis.text.y = element_text(size = 9),
        legend.position = "bottom")
} else {
  cat("Nenhuma via KEGG enriquecida para gerar o barplot.\n")
}
```



9.7.7 Como interpretar os gráficos:

- **Dotplot:** cada ponto é uma via enriquecida, separada nos painéis **activated** ($NES > 0$) e **suppressed** ($NES < 0$). O **tamanho** indica o número de genes da via (**setSize**); a **cor** indica o p-valor ajustado.
- **Ridgeplot:** a curva de densidade mostra como os $\log_2FC$ dos genes daquela via estão distribuídos — concentração à direita = UP, à esquerda = DOWN.
- **Running ES:** a curva clássica do GSEA. Picos no topo do ranking (esquerda) indicam vias UP; vales na base (direita), vias DOWN. As marcas (|) mostram onde os genes da via caem no ranking.
- **Barplot bidirecional (NES):** análogo direto do barplot do ORA, mas usando NES como métrica de direção e magnitude — barras vermelhas (UP) à direita, azuis (DOWN) à esquerda.

9.8 6. Enriquecimento GO com g:Profiler

Vamos rodar a ORA com Gene Ontology (Biological Process) separadamente para cada conjunto de genes (upregulados, downregulados e todos os DEGs), usando o **corte didático** de p-valor ajustado **0.5** com correção **FDR** (Benjamini-Hochberg). Observe que diferentemente do `enrichKEGG()`, aqui passamos os identificadores no formato original do Ensembl Fungi (ex: `C1_00210C_A`) — o g:Profiler reconhece e mapeia internamente, sem precisar de dicionário de conversão.

9.9 7. Sobre o pacote g:Profiler

Uma alternativa para o enriquecimento usando o Gene Ontology é o **g:Profiler**, uma ferramenta mantida pela Universidade de Tartu que indexa anotações funcionais do **Ensembl** e do **Ensembl Genomes** (incluindo o Ensembl Fungi) e oferece uma API REST consultável a partir do R via o pacote `gprofiler2`. A função central do pacote é `gost()`, que envia uma lista de genes ao servidor, executa o teste hipergeométrico contra cada termo da ontologia e retorna uma tabela com os resultados.

A principal vantagem do g:Profiler é a **reprodutibilidade por release**: cada versão do banco recebe um identificador citável (ex: `e114_eg62_p19`) e fica permanentemente arquivada, o que permite que análises sejam refeitas exatamente como na data original.

Ponto Importante — g:Profiler e Organismos Não-Modelo

O g:Profiler suporta **mais de 200 organismos** via Ensembl e Ensembl Genomes, incluindo *C. albicans* SC5314 (código de organismo: `calbicans`). No entanto, a integração com **KEGG, Reactome e WikiPathways** está disponível apenas para organismos-modelo (humano, camundongo, rato, levedura, etc.). Para *C. albicans*, somente as três sub-ontologias do Gene Ontology (**Biological Process**, **Molecular Function** e **Cellular Component**) estão indexadas. Por isso, mantemos a análise KEGG da seção anterior e usamos o g:Profiler aqui apenas para GO (Biological Process).

9.9.1 Carregando o pacote gprofiler2

```
suppressPackageStartupMessages({  
  library(gprofiler2)  
  library(stringr)  
})
```

```

gost_up <- gost(
  query          = genes_up,
  organism       = "calbicans",
  sources        = "GO:BP",
  user_threshold = 0.5,
  correction_method = "fdr"
)

gost_down <- gost(
  query          = genes_down,
  organism       = "calbicans",
  sources        = "GO:BP",
  user_threshold = 0.5,
  correction_method = "fdr"
)

gost_all <- gost(
  query          = genes_all,
  organism       = "calbicans",
  sources        = "GO:BP",
  user_threshold = 0.5,
  correction_method = "fdr"
)

# Versão do banco consultado (citável nos métodos)
cat("Versão do g:Profiler:", gost_all$meta$version, "\n\n")

```

Versão do g:Profiler: e114_eg62_p19_27110d83

```
cat("Termos GO BP enriquecidos:\n")
```

Termos GO BP enriquecidos:

```
cat("  UP      :", if (is.null(gost_up)) 0 else nrow(gost_up$result), "\n")
```

UP : 149

```
cat("  DOWN    :", if (is.null(gost_down)) 0 else nrow(gost_down$result), "\n")
```

DOWN : 143

```
cat(" ALL :", if (is.null(gost_all)) 0 else nrow(gost_all$result), "\n")
```

```
ALL : 240
```

Como o objeto retornado pelo `gost()` é uma lista (não um `enrichResult` do `clusterProfiler`), as funções `dotplot()` e `barplot()` do `enrichplot` não funcionam diretamente. Definimos abaixo uma pequena função auxiliar que monta dotplots no mesmo estilo dos da seção KEGG: o eixo X traz o **GeneRatio** (proporção de DEGs caindo no termo), a **cor** representa o p-valor ajustado e o **tamanho do ponto** indica quantos DEGs pertencem ao termo (Count).

```
# Função para criar o gráfico
dotplot_gost <- function(gost_res, titulo, cor_low, cor_high) {
  df <- gost_res$result %>%
    mutate(
      GeneRatio = intersection_size / query_size,
      term_lbl = str_wrap(term_name, 45)
    ) %>%
    arrange(p_value) %>%
    head(15) %>%
    mutate(term_lbl = factor(term_lbl,
                             levels = unique(term_lbl[order(GeneRatio)])))

  ggplot(df, aes(x = GeneRatio, y = term_lbl,
                 color = p_value, size = intersection_size)) +
    geom_point() +
    scale_color_gradient(low = cor_low, high = cor_high,
                        name = "p-valor\najustado") +
    scale_size_continuous(name = "Count", range = c(3, 8)) +
    labs(title = titulo, x = "GeneRatio", y = NULL) +
    theme_minimal(base_size = 11) +
    theme(plot.title = element_text(face = "bold", hjust = 0.5),
          axis.text.y = element_text(size = 9))
}
```

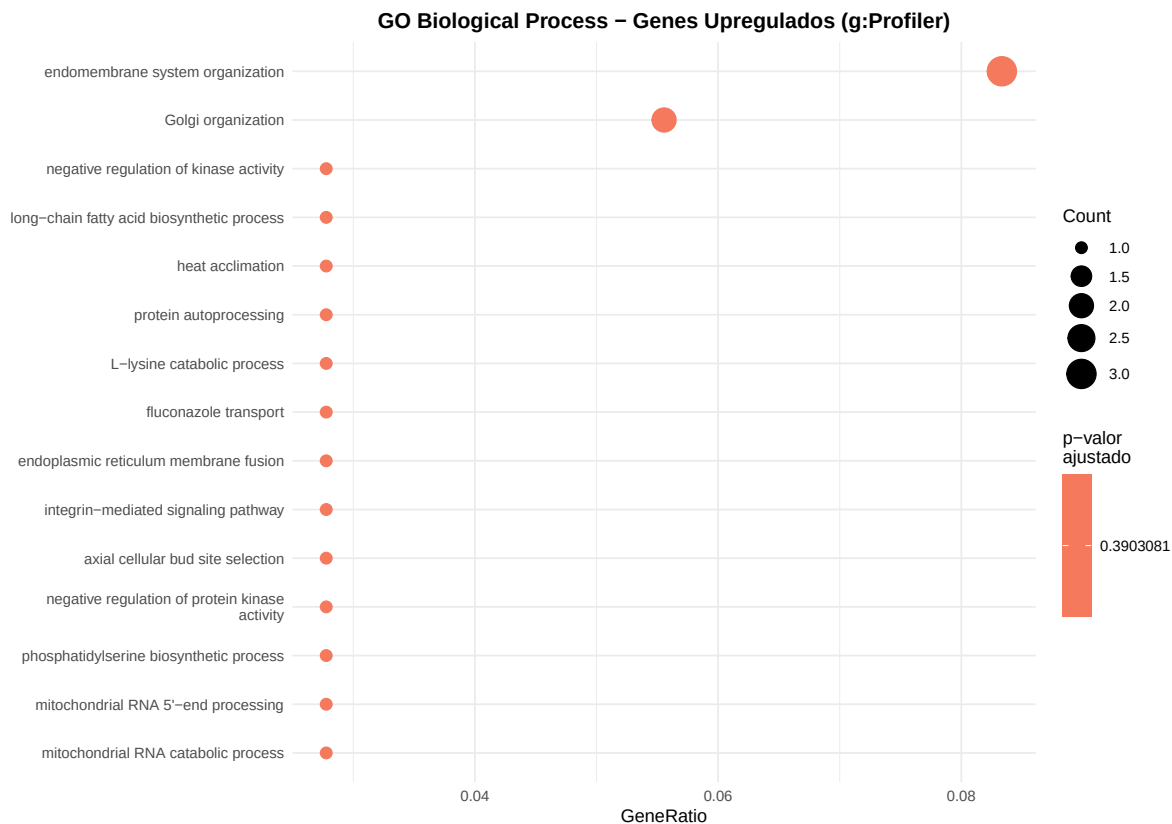
9.9.2 Dotplot — GO BP Upregulados

```
if (!is.null(gost_up) && nrow(gost_up$result) > 0) {
  dotplot_gost(gost_up,
    "GO Biological Process - Genes Upregulados (g:Profiler)",
    cor_low = "#E31A1C", cor_high = "#FCBBA1")
}
```

```

} else {
  cat("Nenhum processo biológico enriquecido para genes upregulados.\n")
}

```

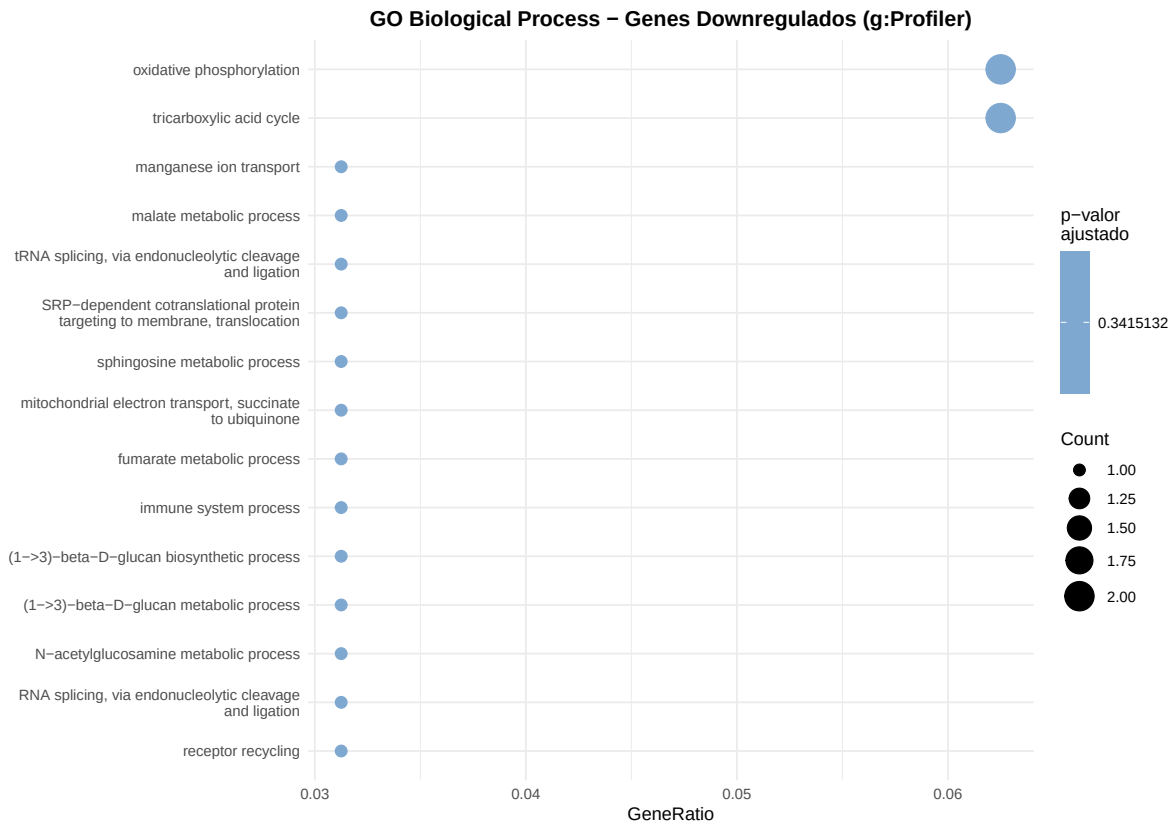


9.9.3 Dotplot — GO BP Downregulados

```

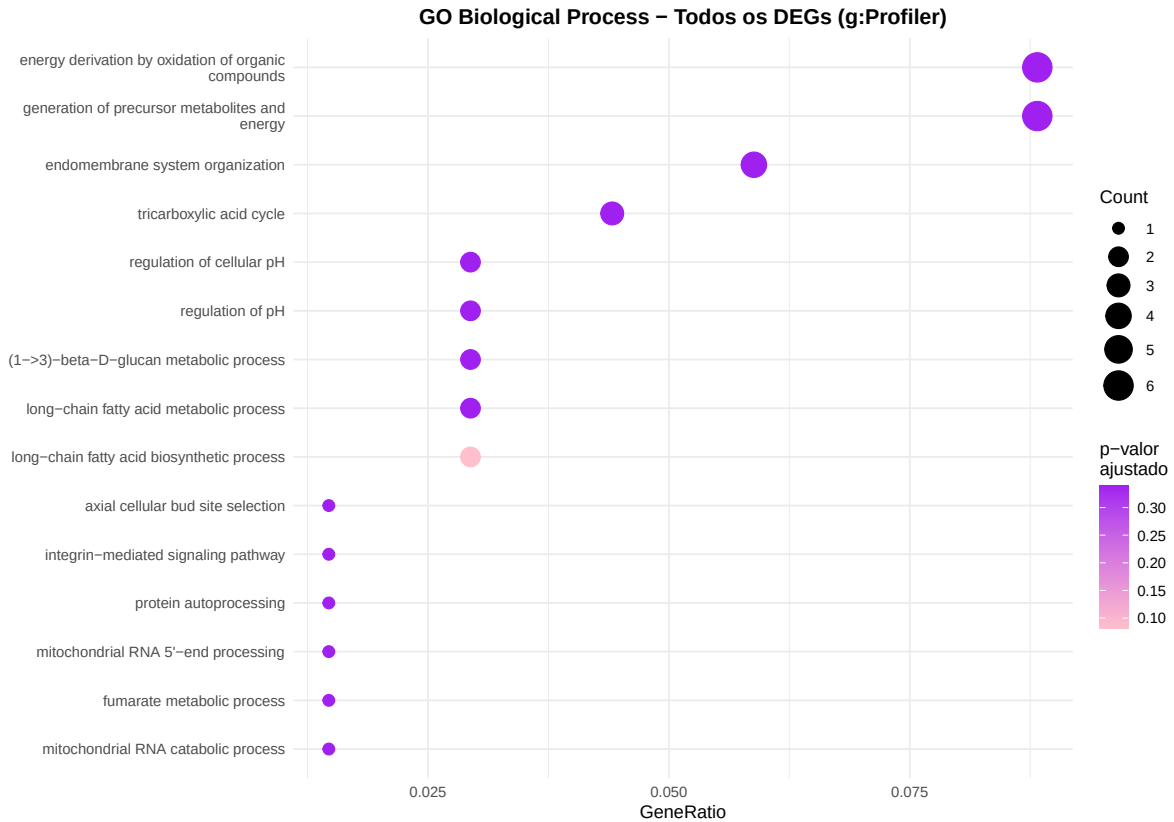
if (!is.null(gost_down) && nrow(gost_down$result) > 0) {
  dotplot_gost(gost_down,
    "GO Biological Process - Genes Downregulados (g:Profiler)",
    cor_low = "#1F78B4", cor_high = "#C6DBEF")
} else {
  cat("Nenhum processo biológico enriquecido para genes downregulados.\n")
}

```



9.9.4 Dotplot — GO BP Todos os DEGs

```
if (!is.null(gost_all) && nrow(gost_all$result) > 0) {
  dotplot_gost(gost_all,
    "GO Biological Process - Todos os DEGs (g:Profiler)",
    cor_low = "pink", cor_high = "purple")
} else {
  cat("Nenhum processo biológico enriquecido para todos os DEGs.\n")
}
```



9.9.5 Barplot Bidirecional — GO BP UP vs. DOWN

Como fizemos na seção KEGG, podemos comparar diretamente os termos enriquecidos em UP e DOWN num único gráfico bidirecional — as barras vermelhas (UP) crescem para a direita e as azuis (DOWN) para a esquerda, compartilhando o mesmo eixo Y:

```
df_gost_up <- if (!is.null(gost_up) && nrow(gost_up$result) > 0) {
  gost_up$result %>%
    mutate(Direction = "Upregulated", Count_dir = intersection_size)
} else { data.frame() }

df_gost_down <- if (!is.null(gost_down) && nrow(gost_down$result) > 0) {
  gost_down$result %>%
    mutate(Direction = "Downregulated", Count_dir = -intersection_size)
} else { data.frame() }

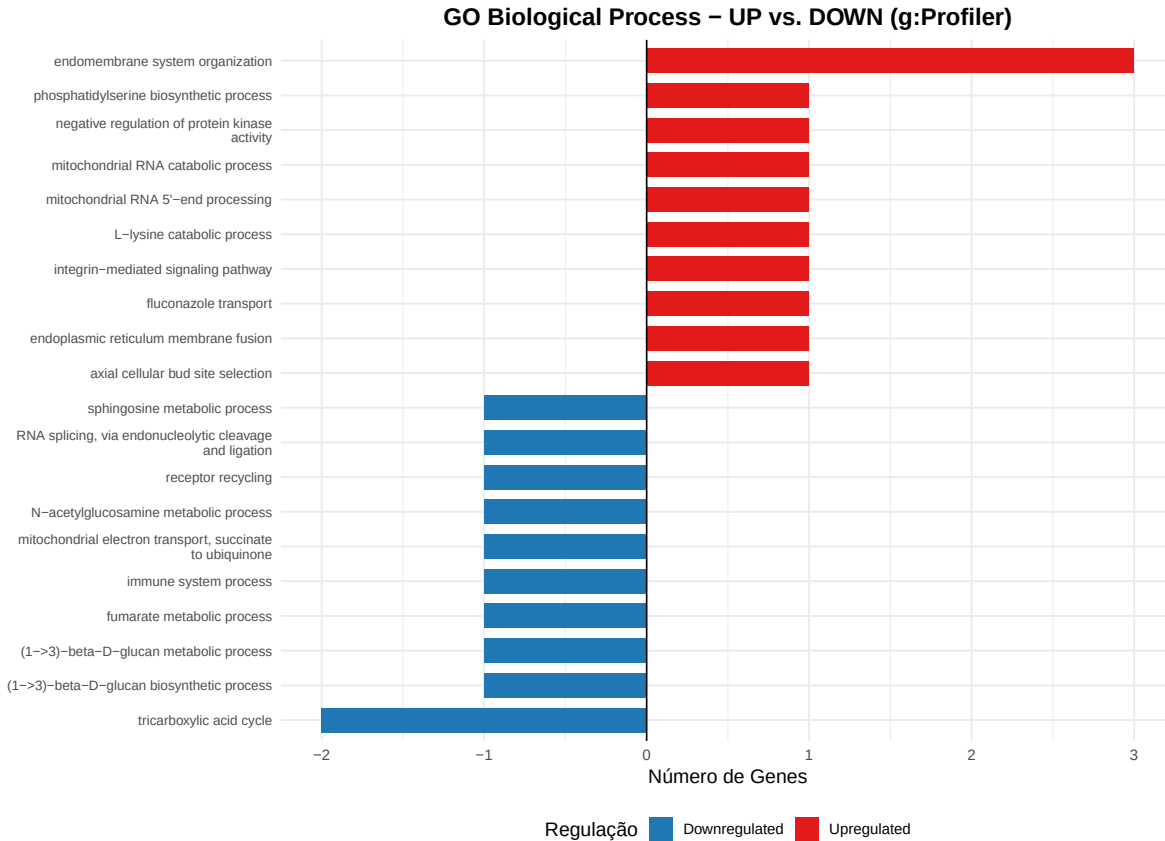
if (nrow(df_gost_up) > 0 || nrow(df_gost_down) > 0) {
  df_gost_bidir <- bind_rows(
```

```

df_gost_up    %>% arrange(p_value) %>% head(10),
df_gost_down %>% arrange(p_value) %>% head(10)
) %>%
  mutate(term_lbl = str_wrap(term_name, 45))

ggplot(df_gost_bidir,
  aes(x = Count_dir, y = reorder(term_lbl, Count_dir), fill =
    ↪ Direction)) +
  geom_col(width = 0.7) +
  scale_fill_manual(values = c("Upregulated" = "#E31A1C",
    "Downregulated" = "#1F78B4")) +
  geom_vline(xintercept = 0, linewidth = 0.5) +
  labs(title = "GO Biological Process - UP vs. DOWN (g:Profiler)",
    x = "Número de Genes", y = NULL, fill = "Regulação") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
    axis.text.y = element_text(size = 9),
    legend.position = "bottom")
} else {
  cat("Nenhum processo biológico enriquecido para gerar o barplot
    ↪ bidirecional.\n")
}

```



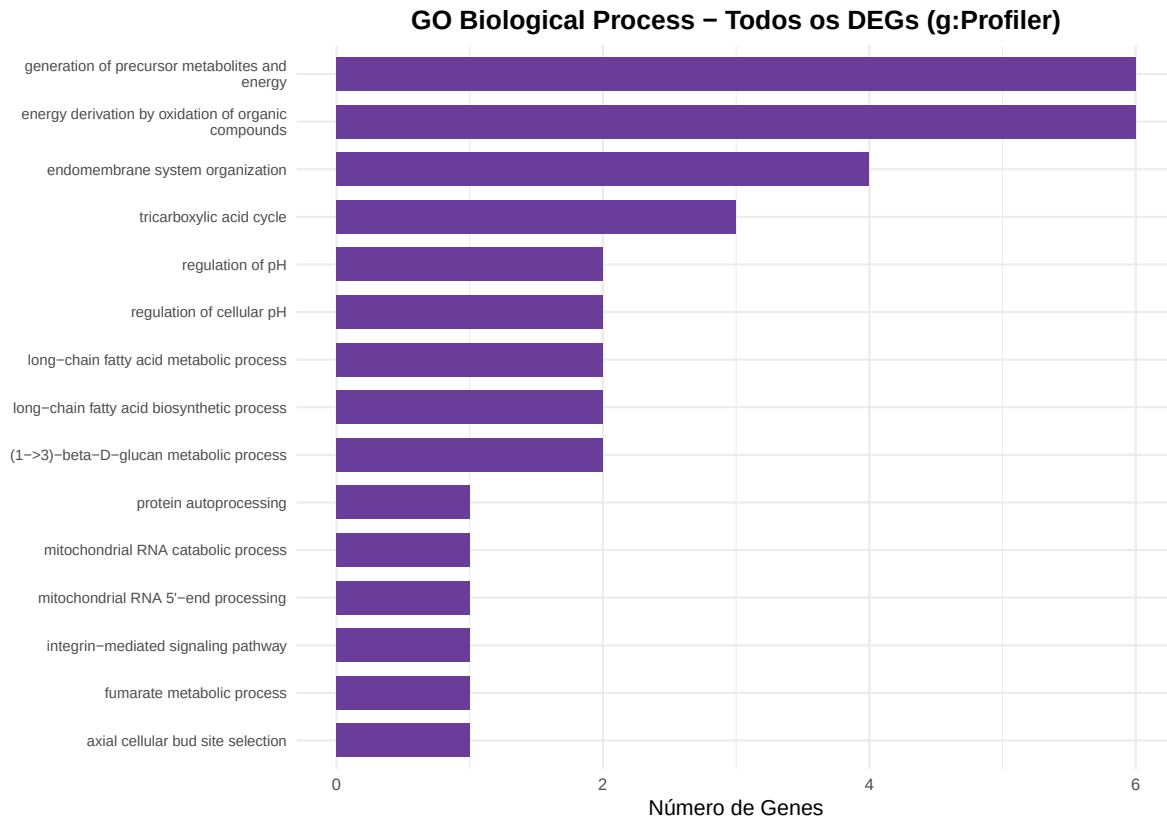
9.9.6 Barplot — GO BP Todos os DEGs

Finalmente, o barplot considerando todos os DEGs juntos, em tom roxo para diferenciá-lo do par UP/DOWN:

```
if (!is.null(gost_all) && nrow(gost_all$result) > 0) {
  gost_all$result %>%
    arrange(p_value) %>%
    head(15) %>%
    mutate(term_lbl = str_wrap(term_name, 45)) %>%
    ggplot(aes(x = intersection_size,
               y = reorder(term_lbl, intersection_size))) +
    geom_col(fill = "#6A3D9A", width = 0.7) +
    labs(title = "GO Biological Process - Todos os DEGs (g:Profiler)",
         x = "Número de Genes", y = NULL) +
    theme_minimal(base_size = 13) +
    theme(plot.title = element_text(hjust = 0.5, face = "bold"),
```



```
axis.text.y = element_text(size = 9))
} else {
  cat("Nenhum processo biológico enriquecido para todos os DEGs.\n")
}
```



9.9.7 Como interpretar os gráficos:

- **Dotplot:** cada ponto representa um termo GO BP enriquecido. O **tamanho** indica quantos DEGs pertencem ao termo (Count); a **cor** indica o p-valor ajustado (FDR).
- **Barplot bidirecional:** permite comparar diretamente os termos UP (vermelho, à direita) e DOWN (azul, à esquerda) em um único gráfico, com o eixo Y compartilhado.
- **Barplot (ALL):** mostra os termos GO BP enriquecidos considerando todos os DEGs juntos, ordenados pelo número de genes que pertencem a cada termo.

9.10 8. Resumo e Interpretação

```
cat("=== Resumo do Enriquecimento Funcional ===\n\n")
```

```
=== Resumo do Enriquecimento Funcional ===
```

```
cat("Genes UP:", length(genes_up), "| DOWN:", length(genes_down),  
    "| Total:", length(genes_all), "\n\n")
```

```
Genes UP: 40 | DOWN: 35 | Total: 75
```

```
cat("--- KEGG (GSEA) ---\n")
```

```
--- KEGG (GSEA) ---
```

```
if (!is.null(gsea_kegg) && nrow(as.data.frame(gsea_kegg)) > 0) {  
  df_gsea_sum <- as.data.frame(gsea_kegg)  
  cat("  Activated (NES > 0):", sum(df_gsea_sum$NES > 0), "vias\n")  
  cat("  Suppressed (NES < 0):", sum(df_gsea_sum$NES < 0), "vias\n")  
  cat("  Total                :", nrow(df_gsea_sum), "vias\n")  
} else {  
  cat("  nenhuma via enriquecida\n")  
}
```

```
Activated (NES > 0): 26 vias  
Suppressed (NES < 0): 36 vias  
Total                : 62 vias
```

```
cat("\n--- GO Biological Process (g:Profiler) ---\n")
```

```
--- GO Biological Process (g:Profiler) ---
```

```
cat("  UP:  ")
```

```
UP:
```

```
if (!is.null(gost_up) && nrow(gost_up$result) > 0) {  
  cat(nrow(gost_up$result), "termos\n")  
} else { cat("nenhum termo\n") }
```

149 termos

```
cat("  DOWN: ")
```

DOWN:

```
if (!is.null(gost_down) && nrow(gost_down$result) > 0) {  
  cat(nrow(gost_down$result), "termos\n")  
} else { cat("nenhum termo\n") }
```

143 termos

```
cat("  ALL: ")
```

ALL:

```
if (!is.null(gost_all) && nrow(gost_all$result) > 0) {  
  cat(nrow(gost_all$result), "termos\n")  
} else { cat("nenhum termo\n") }
```

240 termos

Identificamos quais vias biológicas estão enriquecidas entre os genes diferencialmente expressos da *Candida albicans*. Mas como esses genes se relacionam entre si? Será que as proteínas codificadas por eles interagem fisicamente ou funcionam nos mesmos processos? Para responder a essas perguntas, vamos construir uma rede de interação proteína-proteína (PPI).

10 Redes de Interação Proteína-Proteína (PPI)

Uma lista de genes diferencialmente expressos nos diz **quais** genes mudaram. O enriquecimento funcional nos diz **em quais vias** esses genes participam. Mas nenhuma dessas análises nos mostra **como** os produtos desses genes se relacionam entre si. É exatamente essa lacuna que as redes de interação proteína-proteína (PPI) preenchem.

Uma rede PPI é um grafo onde cada **nó** representa uma proteína e cada **aresta** conecta duas proteínas que possuem alguma relação funcional ou física documentada. Ao mapear nossos genes diferencialmente expressos nessa rede.

10.1 9. O Banco de Dados STRING

O [STRING \(Search Tool for the Retrieval of Interacting Genes/Proteins\)](#) é o maior banco de dados público de interações proteicas, cobrindo mais de 14.000 organismos. Um ponto crucial é entender que o STRING **não armazena apenas interações físicas diretas** (como ligações proteína-proteína demonstradas por co-imunoprecipitação). Ele integra **múltiplas fontes de evidência** para inferir **associações funcionais** entre proteínas:

Canal de Evidência	Coluna	O que mede
Vizinhança genômica	nscore	Genes fisicamente próximos no genoma tendem a ser funcionalmente relacionados
Fusão gênica	fscore	Genes que aparecem fundidos em outros organismos provavelmente interagem
Co-ocorrência filogenética	pscore	Proteínas que co-evoluem (presentes/ausentes juntas em espécies)
Co-expressão	ascore	Genes com padrões de expressão similares em múltiplos experimentos
Evidência experimental	escore	Interações demonstradas por ensaios bioquímicos (Y2H, AP-MS, etc.)

Canal de Evidência	Coluna	O que mede
Bancos de dados curados	dscore	Interações registradas em bancos curados (KEGG, Reactome, BioCyc, etc.)
Text mining	tscore	Co-menção de proteínas na literatura científica

Cada canal produz um **escore** entre 0 e 1 que indica a confiança daquela evidência para um dado par de proteínas. O STRING combina esses escores em um **escore combinado** usando uma fórmula probabilística que evita dupla contagem.

! Interações Interações Físicas

É fundamental entender que o termo “interação” no STRING é mais amplo do que interação física direta. Quando dizemos que duas proteínas “interagem” no STRING, estamos dizendo que há **evidência de associação funcional**, elas podem participar do mesmo complexo, da mesma via metabólica, ou serem co-reguladas, sem necessariamente se tocarem fisicamente. Por isso, o termo mais preciso é **associação funcional**.

i Ponto Importante — Bancos de Dados Curados e Organismos Não-Modelo

Assim como vimos na análise de enriquecimento funcional, a qualidade dos resultados de redes PPI depende diretamente da **completude do banco de dados para o organismo estudado**. Para organismos-modelo como *Homo sapiens* e *Saccharomyces cerevisiae*, o STRING possui milhares de interações experimentalmente validadas. Para *Candida albicans*, a cobertura é menor, muitas interações são inferidas computacionalmente a partir de ortologia com *S. cerevisiae* ou por text mining. Mesmo assim, o STRING suporta *C. albicans* SC5314 nativamente (identificador taxonômico **237561**), o que nos permite consultar a base diretamente.

10.2 10. Carregando os Pacotes

```
suppressPackageStartupMessages({
  library(RCurl)
  library(igraph)
  library(ggraph)
  library(ggplot2)
  library(dplyr)
```

```
library(readr)
library(tibble)
library(scales)
})
```

10.3 11. Preparando a Lista de Genes

Vamos recuperar nossos resultados de expressão diferencial e preparar a lista de genes que será submetida ao STRING. O STRING não reconhece diretamente os identificadores do Ensembl Fungi (ex: C1_00210C_A). Para *C. albicans*, o STRING utiliza os identificadores no formato CAALFM_ (ex: CAALFM_C100210CA), que são os mesmos usados pelo CGD e pelo KEGG. Felizmente, já construímos o dicionário de conversão (`ensembl_to_kegg`) na seção anterior:

```
# Carregar resultados da expressão diferencial
res_dge <- readRDS(here::here("data/DGE/res_dge.rds"))

# Selecionar apenas os genes significativos (DEGs)
res_dge_signif <- res_dge %>%
  filter(signif != "Not significant")

genes_deg <- rownames(res_dge_signif)

# Converter Ensembl IDs para o formato CAALFM_ usando o dicionário da seção 4
genes_deg_caalfm <- unname(ensembl_to_kegg[genes_deg[genes_deg %in%
  ↪ names(ensembl_to_kegg)])])

# Criar dicionário reverso para mapear de volta
caalfm_to_ensembl <- setNames(names(ensembl_to_kegg),
  ↪ unname(ensembl_to_kegg))

cat("Total de DEGs:", length(genes_deg), "\n")
```

Total de DEGs: 75

```
cat("DEGs com ID CAALFM_ mapeado:", length(genes_deg_caalfm), "\n")
```

DEGs com ID CAALFM_ mapeado: 75

i Ponto Importante — Conversão de Identificadores

A conversão entre Ensembl Fungi e CAALFM_ segue uma regra simples: o ID C1_00010W_A corresponde a CAALFM_C100010WA (remoção dos underscores e adição do prefixo). Essa é a mesma conversão que fizemos para o KEGG na seção 4. Se um gene não possui correspondência no dicionário, ele simplesmente não será incluído na rede.

10.4 12. Consultando o STRING via API

O STRING disponibiliza uma [API pública](#) que permite realizar consultas programáticas. Vamos utilizá-la para: (1) mapear nossos identificadores CAALFM_ para os identificadores internos do STRING, e (2) obter a rede de interações entre eles.

O identificador taxonômico da cepa SC5314 de *C. albicans* no STRING é **237561**.

10.4.1 Mapeando os identificadores

```
# Concatenar os genes no formato CAALFM_ para a requisição à API
genes_concatenado <- paste0(genes_deg_caalfm, collapse = "%0d")

# Requisição para mapear nossos IDs para os IDs do STRING
req_map <- RCurl::postForm(
  "https://string-db.org/api/tsv/get_string_ids",
  identifiers = genes_concatenado,
  echo_query = "1",
  species = "237561" # C. albicans SC5314
)

map_ids <- read.table(text = req_map, sep = "\t", header = TRUE, quote = "")

cat("Genes mapeados no STRING:", nrow(map_ids), "de",
    ↪ length(genes_deg_caalfm), "\n")
```

Genes mapeados no STRING: 70 de 75

10.4.2 Obtendo a rede de interações

```
# Concatenar os identificadores do STRING
genes_string_concatenado <- paste0(unique(map_ids$stringId), collapse =
  ↪ "%0d")

# Requisição para o método 'network'
req_net <- RCurl::postForm(
  "https://string-db.org/api/tsv/network",
  identifiers = genes_string_concatenado,
  required_score = "0", # Sem filtro mínimo (filtraremos depois)
  species = "237561"
)

int_network <- read.table(text = req_net, sep = "\t", header = TRUE)
int_network <- unique(int_network)

cat("Interações brutas obtidas:", nrow(int_network), "\n")
```

Interações brutas obtidas: 237

10.5 13. Filtrando as Interações com combinescores

As interações brutas incluem todas as fontes de evidência e todos os níveis de confiança. Para obtermos uma rede de alta qualidade, precisamos:

1. **Selecionar os canais de evidência** mais confiáveis para o nosso contexto.
2. **Combinar os escores** desses canais usando a fórmula probabilística do STRING.
3. **Filtrar** pelo escore combinado mínimo.

A função `combinescores` abaixo implementa exatamente esse procedimento. Ela recebe a tabela de interações, seleciona os canais desejados e calcula o escore combinado usando a fórmula: $1 - \prod_i (1 - S_i)$, onde S_i é o escore de cada canal. Essa abordagem probabilística garante que **múltiplas fontes fracas de evidência possam se somar** para produzir uma interação de alta confiança.

```
# Função para combinar escores de acordo com o algoritmo do STRING
combinescores <- function(dat, evidences = "all", confLevel = 0.4) {
  if (evidences[1] == "all") {
    edat <- dat[, -c(1, 2, ncol(dat))]
  } else {
    if (!all(evidences %in% colnames(dat))) {
      stop("ERRO: uma ou mais evidências não encontradas nas colunas dos
        ↪ dados!")
    }
  }
}
```



```

    }
    edat <- dat[, evidences]
  }
  # Os escores do STRING vêm em escala 0-1000; converter para 0-1
  if (any(edat > 1)) {
    edat <- edat / 1000
  }
  # Fórmula probabilística: 1 - prod(1 - Si)
  edat <- 1 - edat
  sc <- apply(X = edat, MARGIN = 1, FUN = function(x) 1 - prod(x))
  dat <- cbind(dat[, c(1, 2)], combined_score = sc)
  idx <- dat$combined_score >= confLevel
  dat <- dat[idx, ]
  return(dat)
}

```

Vamos aplicar o filtro. Utilizaremos os canais de **co-expressão**, **evidência experimental** e **bancos de dados curados**. Aqui, por motivos didáticos, vamos estabelecer um escore combinado mínimo de **0.15** (baixa confiança):

```

# Aplicar o filtro por canais e confiança mínima
int_network_filtered <- combinescores(
  int_network,
  evidences = c("ascore", "escore", "dscore"),
  confLevel = 0.15
)

cat("Interações após filtro (confiança 0.15):", nrow(int_network_filtered),
    "\n")

```

Interações após filtro (confiança 0.15): 157

i Ponto Importante — Escolha do Threshold

O STRING utiliza uma classificação de confiança para seus escores:

- **Low confidence:** > 0.15
- **Medium confidence:** > 0.4
- **High confidence:** > 0.7
- **Highest confidence:** > 0.9

Para *C. albicans*, um organismo com menos interações experimentalmente validadas, utilizamos o threshold de **baixa confiança (0.15)** de forma didática para obter uma rede

com conexões suficientes a fim de explorar e interpretar grafos durante a aula. Diferentes escores devem ser empregados (como > 0.4 ou 0.7) a depender do contexto e perguntas biológicas com intenção em dados com nível maior de acurácia.

10.6 14. Construindo o Grafo com igraph

Agora vamos transformar a tabela de interações filtrada em um **objeto igraph** e anotar cada nó com as informações de expressão diferencial (log2FC e significância):

```
# Criar dataframe de anotação mapeando stringId → nomes preferenciais e IDs
↪ originais
ann <- data.frame(
  stringId = map_ids$stringId,
  gene_name = map_ids$preferredName,
  query_caalfm = map_ids$queryItem,
  stringsAsFactors = FALSE
) %>%
distinct(stringId, .keep_all = TRUE) %>%
mutate(
  # Mapear CAALFM_ de volta para Ensembl ID
  ensembl_id = caalfm_to_ensembl[query_caalfm]
)

# Substituir os stringIds pelos nomes dos genes na tabela de interações
int_network_named <- int_network_filtered %>%
mutate(
  gene_A = ann$gene_name[match(stringId_A, ann$stringId)],
  gene_B = ann$gene_name[match(stringId_B, ann$stringId)]
) %>%
filter(!is.na(gene_A), !is.na(gene_B)) %>%
dplyr::select(gene_A, gene_B, combined_score)

# Criar objeto igraph (rede não direcionada)
g <- graph_from_data_frame(int_network_named, directed = FALSE)

# Remover arestas duplicadas e auto-loops
g <- igraph::simplify(g)

cat("Rede construída:", vcount(g), "nós,", ecount(g), "arestas\n")
```

Rede construída: 64 nós, 157 arestas

10.6.1 Anotando os nós com informações de expressão

```
# Criar dicionário gene_name → ensembl_id
name_to_ensembl <- setNames(ann$ensembl_id, ann$gene_name)

# Para cada nó do grafo, buscar o Ensembl ID e depois as informações do DEG
node_names <- V(g)$name
node_ensembl <- name_to_ensembl[node_names]

# Extrair log2FC e padj
node_log2fc <- res_dge[node_ensembl, "log2FoldChange"]
node_padj <- res_dge[node_ensembl, "padj"]
node_signif <- res_dge[node_ensembl, "signif"]

# Atribuir como atributos do grafo
V(g)$log2FC <- ifelse(is.na(node_log2fc), 0, node_log2fc)
V(g)$padj <- ifelse(is.na(node_padj), 1, node_padj)
V(g)$signif <- ifelse(is.na(node_signif), "Not significant", node_signif)
V(g)$ensembl_id <- ifelse(is.na(node_ensembl), "", node_ensembl)

# Calcular -log10(padj) como atributo (usado na visualização)
V(g)$neg_log10_padj <- -log10(ifelse(V(g)$padj == 0, .Machine$double.xps,
↪ V(g)$padj))
```

10.7 15. Layout Interativo com easylayout

Para posicionar os nós da rede de forma visualmente clara, utilizaremos o pacote [easylayout](#), desenvolvido pelo nosso grupo. O easylayout é um pacote R que utiliza **simulações de forças interativas** diretamente dentro do RStudio, permitindo ajustar o layout em tempo real antes de exportar o resultado para plotagem com ggraph ou ggplot2.

O easylayout **não** é uma biblioteca de visualização, ele é uma ferramenta de **posicionamento** que se integra com bibliotecas existentes. Ele recebe um objeto igraph, abre uma aplicação web interativa dentro do IDE, e retorna uma **matriz de coordenadas XY** que pode ser usada por qualquer pacote de plotagem.

Usando o easylayout na prática

1. Execute `layout <- easylayout(g)` — uma janela interativa abrirá no RStudio.
2. Ajuste as forças de atração/repulsão em tempo real até que o layout fique satisfatório.

3. Use o modo de edição para mover nós manualmente, se necessário.
4. Clique em “Export” — as coordenadas serão enviadas de volta ao R.
5. Plote com `ggraph(g, layout = layout) + ...`

```
library(easylayout)

# Layout interativo(Descomente a linha para executar) - uma janela abrirá no
↪ RStudio
# layout_ppi <- easylayout(g)

# Salvar o layout para reprodutibilidade
# saveRDS(layout_ppi, file = here::here("data/DGE/layout_ppi.rds"))
```

10.8 16. Visualização da Rede PPI com ggraph

Vamos plotar a rede utilizando `ggraph`, colorindo os nós de acordo com o **log2 Fold Change** (escala contínua: azul para downregulados, vermelho para upregulados) e ajustando o tamanho conforme a **significância estatística** ($-\log_{10}$ do p-valor ajustado):

```
# Carregar layout

layout_ppi <- readRDS(here::here("data/DGE/layout_ppi.rds"))

ggraph(g, layout = layout_ppi) +
  geom_edge_link(alpha = 0.3, color = "grey60", width = 0.3) +
  geom_node_point(
    aes(color = log2FC, size = neg_log10_padj),
    alpha = 0.85
  ) +
  geom_node_text(
    aes(label = name),
    size = 2.8,
    repel = TRUE,
    max.overlaps = 20,
    color = "black",
    segment.color = "grey50",
    segment.size = 0.3
  ) +
  scale_color_gradient2(
    low = "#1F78B4",
```

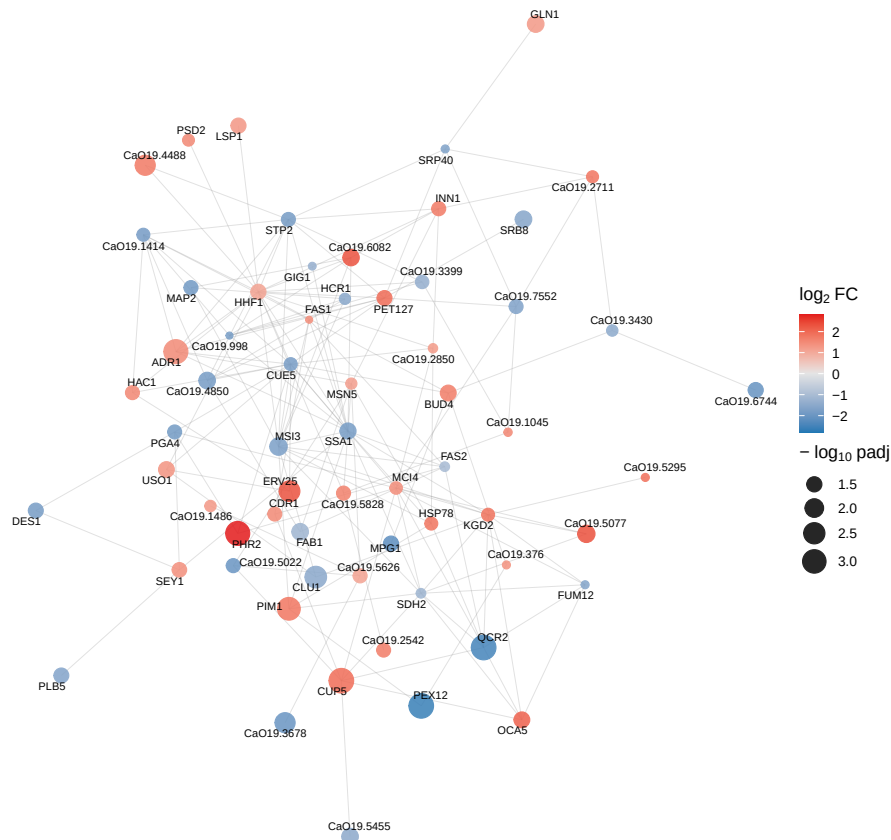
```

mid = "grey90",
high = "#E31A1C",
midpoint = 0,
name = expression(log[2] ~ "FC"),
limits = c(
  -max(abs(V(g)$log2FC), na.rm = TRUE),
  max(abs(V(g)$log2FC), na.rm = TRUE)
)
) +
scale_size_continuous(
  name = expression("-" ~ log[10] ~ "padj"),
  range = c(2, 8)
) +
labs(
  title = "Rede PPI - Genes Diferencialmente Expressos de C. albicans",
  subtitle = "Interações STRING (co-expressão + experimental + banco de
    ↪ dados, score 0.4)",
  caption = "Fonte: STRING v12 | Organismo: C. albicans SC5314 (taxid:
    ↪ 237561)"
) +
coord_fixed() +
theme_void(base_size = 13) +
theme(
  plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
  plot.subtitle = element_text(hjust = 0.5, color = "grey40", size = 10),
  plot.caption = element_text(hjust = 1, color = "grey50", size = 8),
  legend.position = "right",
  plot.margin = margin(10, 10, 10, 10)
)

```

Rede PPI – Genes Diferencialmente Expressos de *C. albicans*

Interações STRING (co-expressão + experimental + banco de dados, score ≥ 0.4)



Fonte: STRING v12 | Organismo: *C. albicans* SC5314 (taxid: 237561)

10.8.1 Como interpretar a rede PPI:

- **Cor dos nós:** segue uma escala contínua do **azul** (downregulado sob vancomicina) ao **vermelho** (upregulado), passando por cinza (log₂FC próximo de zero). Isso permite identificar rapidamente se regiões da rede são dominadas por up ou downregulação.
- **Tamanho dos nós:** proporcional a $-\log_{10}(\text{padj})$ quanto maior o ponto, mais significativa é a mudança de expressão daquele gene.
- **Arestas:** representam associações funcionais documentadas no STRING (co-expressão, evidência experimental ou bancos de dados curados). Arestas mais densas indicam sub-redes de proteínas que atuam de forma coordenada.
- **Rótulos:** aparecem apenas para os 10% dos genes mais significativos, destacando os candidatos de maior interesse biológico.

- **Clusters visuais:** grupos de nós densamente conectados sugerem módulos funcionais — conjuntos de proteínas que trabalham juntas em um processo biológico. Identifique se esses módulos correspondem às vias enriquecidas que encontramos anteriormente.

*Concluimos a análise completa de RNA-seq da Candida albicans** em resposta à vancomicina. Desde a descontaminação das leituras do hospedeiro com o nf-core/detaxizer, passando pela quantificação com o nf-core/rnaseq, a análise de expressão diferencial com DESeq2, o enriquecimento funcional com KEGG e GO, até a construção e visualização de redes de interação proteína-proteína com o STRING — percorremos um pipeline completo e reproduzível para transcriptômica de patógenos, enfrentando os desafios específicos de um organismo não-modelo.*